

Design and Development of Flight Controller for Omnidirectional Airship Drones

Pala Ahmed

Master Thesis

2022

The work was carried out at roboloon, Stuttgart



University of Stuttgart
Institute of Flight Mechanics und Control

Abstract

Unmanned Aerial Vehicles (UAVs) have received a lot of attention in research and in civil and commercial applications. There is a strong focus on multirotors and fixed-wing UAVs, while lighter-than-air (LTA) vehicles pose major challenges due to their sluggish maneuverability. In order to take advantage of the significantly higher flight times of airships, a highly maneuverable propulsion system was developed at roboloon, Stuttgart, to allow airships to have all 6 degrees of freedom and overcome the shortcomings of traditional airships. In this work, a flight controller is developed for this omnidirectional airship drone oblX_proto with a novel propulsion system, consisting of nine engines and four rudders, and enabling an instant maneuverability in any direction. The dynamics of the airship is modeled with the Newton-Euler equations of motion and is integrated in the simulation environment of Gazebo as Software-In-The-Loop, using the PX4 flight stack. The flight controller is designed in a modular structure with a cascaded P-PID position and attitude controller and a control allocation, responsible for allocating the control command of the controller to the individual engines. The dynamic control allocation is computing the optimal allocation based on the dynamics of the airship by stating the allocation as a linear optimization problem and solving it with the simplex method. The flight controller is evaluated in several scenarios and flight tasks, validating its performance, resilience and the ability for omnidirectional flight.

Zusammenfassung

Unbemannte Luftfahrzeuge haben eine hohe Popularität in der Forschung und in zivilen und kommerziellen Anwendungen. Dabei liegt im Allgemeinen ein starker Fokus auf Multikopter- und Starrflügler-Drohnen, während Drohnen mit leichter-als-Luft Technologie aufgrund ihrer Trägheit und begrenzten Manövrierfähigkeit große Herausforderungen mit sich bringen. Um die deutlich höheren Flugzeiten von Luftschiffen auszunutzen, wurde bei roboloon, Stuttgart, ein hochmanövrierfähiges Antriebssystem entwickelt, das Luftschiffen die Bewegung in allen 6 Freiheitsgraden ermöglicht und die Mängel herkömmlicher Luftschiffe überwindet. In dieser Arbeit wird ein Flugregler für die omnidirektionale Luftschiff-Drohne oblX_proto mit einem neuartigen Antriebssystem entwickelt, das aus neun Motoren und vier Rudern besteht und eine instantane Manövrierfähigkeit in jede Richtung ermöglicht. Die Dynamik des Luftschiffs wird mit den Newton-Euler-Bewegungsgleichungen modelliert und das Modell als Software-In-The-Loop unter Verwendung der PX4 Software Plattform in die Simulationsumgebung von Gazebo integriert. Der Flugregler hat einen modularen Aufbau und besteht aus einem kaskadierten P-PID Positions- und Lageregler und einer dynamischen Zuweisung des Steuerbefehls des Reglers auf die einzelnen Motoren. Die Zuweisung des ermittelten Steuerbefehls zu den einzelnen Motoren erfolgt unter Berücksichtigung der Dynamik des Luftschiffs und wird als lineares Optimierungsproblem formuliert. Dieses lineare Optimierungsproblem wird kontinuierlich während des Fluges aufgestellt und mit dem Simplex-Verfahren gelöst. Der Flugregler wird in unterschiedlichen Szenarien evaluiert, wobei die Leistung, die Widerstandsfähigkeit und die Fähigkeit zum omnidirektionalen Flug validiert werden.

Contents

1	Introduction	1
1.1	Approach and Structure of the Thesis	2
1.2	Related Work	4
1.3	The oblX_proto Airship Drone	5
2	Fundamentals	7
2.1	Dynamics of the Airship Drone	7
2.1.1	Kinematics and Dynamics	8
2.1.2	Inverse Kinematics and Inverse Dynamics	11
2.2	Linear Optimization	14
2.2.1	Basic Feasible Solutions	15
2.2.2	The Primal Simplex Method	16
3	Software and Simulation Environment	19
3.1	Flight Stack	19
3.2	Simulation Environment	21
3.3	Airship Model	23
3.3.1	Physical Model	24
3.3.2	Aerodynamic Model	25
3.3.3	Motor Model	25
4	Flight Controller	27

4.1	Flight Controller Architecture	28
4.2	Motor Mixer	29
4.2.1	Linear Optimization Problem	31
4.2.2	Input Scaling	34
4.2.3	Software and Hardware Implementation Issues	35
4.3	Control Allocation	35
4.3.1	Linear Optimization Problem	37
4.3.2	Input Scaling	38
4.4	PID Controller	38
4.4.1	Position Controller	38
4.4.2	Attitude Controller	40
5	Evaluation	42
5.1	Validation of the Mixer and Control Allocation	43
5.1.1	Actuation Accuracy	43
5.1.2	Actuation Effectiveness	46
5.2	Validation of the Flight Controller	49
5.2.1	Waypoint Tracking	50
5.2.2	External Force Compensation	53
6	Conclusion	57
	Appendices	60
A.1	Motor Mixer	60

Chapter 1

Introduction

The use of Unmanned Aerial Vehicles (UAVs) in civil and commercial applications has become more prevalent in the last decade due to significant improvements in hardware and software algorithms that enable active control with stabilization, while still being lightweight, energy-efficient and cost-effective. UAVs are particularly suited for operations beyond human reach or where human deployment poses a serious risk. Beside popular applications in media and aerial photography, applications like maintenance in the infrastructure and surveying in areas like agriculture, wildlife and security are becoming more and more common [MazurEtAl16, dro18].

Among the UAV types, fixed-wing and multirotor UAVs have become well-researched subjects in the field of aerial robotics. Quadcopters are the most common type among the UAVs and have low entry barriers due to their simple, transferrable and inexpensive design. There are many comprehensive works to build upon, from designing and modeling quadcopters [Musa18, FurrerEtAl16] to developing controller using even advanced model-based and machine learning-based control methods [HanoverEtAl22, HwangboEtAl17]. In contrast, lighter-than-air (LTA) vehicles such as airships pose significantly more difficult challenges in hardware development, modeling and simulation, and in standardizing a common platform. Unlike quadcopters, ready-to-use open-source flight controller are not available, since the transferability of control algorithms from one vehicle to another is not always given due to differences in size and shape of airships and differences in the propulsion systems.

The LTA technology gives airships a distinct advantage over other UAVs in terms of flight time and safety for maintenance and surveying since LTA vehicles do not have to spend energy to stay in air. However, airships are highly susceptible to wind due to their large volume in relation to their mass. It takes more effort in order to develop a controller that is able to respond to strong disturbances

in a satisfactory manner. This thesis aims to develop a flight controller for an omnidirectional airship drone with a novel propulsion system that allows for an instant maneuverability in every direction and a more stable response to external disturbances. The motion in any of the six degrees of freedom (DOF) is made possible by nine fixed thrusters and four tiltable rudders. By actuating the thrusters in specific combinations, any desired force or torque in any direction can be generated to compensate for disturbances, even from a standing position, while the rudders and the stern engine allow for a more energy-efficient linear flight. Unlike conventional airships, this propulsion system does not require an initial longitudinal velocity to change direction, turn or move the airship laterally, increasing the controllability and stability of the system.

1.1 Approach and Structure of the Thesis

The aim of the thesis is to design a flight controller for the highly maneuverable airship drone prototype `oblX_proto`. The development steps consider the hardware specifications of the airship drone, which are specified in Section 1.3. This thesis follows a practical oriented approach for the development as described in the following five steps:

1. **Flight Stack:** using an open source flight stack can increase the reliability of the software system and autopilot and, moreover, reduce the development time. The setup of the simulation and the development of the flight controller build on the flight stack and its architecture. Therefore, the first step is to evaluate various flight stacks based on the flight controller requirements and compatibility with the flight control hardware, and select a flight stack.
2. **Dynamic model:** UAV flight controller, especially for airships, depend heavily on the accuracy of the dynamic model of the vehicle. To set up the simulation, determine an optimal allocation of control commands, and design a robust flight controller, the dynamic model is set up based on the physical properties of the airship and of the propulsion system. The physical model of the airship is derived from the CAD model. For the model of the propulsion system, thrust tests of the engines were carried out in preliminary studies to measure the behavior and the generated thrust of the engines. For the aerodynamical model, there were no preliminary studies done to determine model parameters. Therefore, the aerodynamical model is simplified to a linear model to reduce the complexity.
3. **Simulation:** the dynamic model of the airship drone is used to setup the simulation. The simulation is used to design and continuously test the flight

controller before it is deployed on the hardware onboard the airship. To simulate real-world operations, a Software-In-The-Loop (SITL) is set up in this work. The SITL bridges the flight stack with the simulation and therefore enables the testing of several features and functionalities of the autopilot including the whole flight controller pipeline.

4. **Optimal motor mixer and control allocation:** with a total of nine engines and four rudders, the airship is an over-actuated system and there are, in theory, infinite ways to actuate the engines in specific combinations to maneuver in a desired direction. In order to achieve an effective control of the airship, the control commands need to be allocated correctly to the respective engines. One way of allocating is to use a static motor mixer that translates the vehicle control commands from the control algorithm to lower level actuator control commands, which is mainly used for manual control. For a more energy-efficient flight, a dynamic control allocation is additionally developed, which enables an optimal allocation at each time step and in any given situation, while considering the dynamics of the system. Both the motor mixer and the control allocation are determined based on the Newton-Euler equations of motion and a linear optimization formulation to minimize the cost of actuation for a desired motion.
5. **Flight control algorithm:** based on the preliminary work, the control method for the flight controller is elaborated and implemented in the selected flight stack. The control allocation provides a simplified interface for the flight control algorithm, reducing the number of control variables to the achievable degrees of freedom. For the oblX_proto, a cascaded P-PID controller is implemented, which is split into a position and an attitude controller. The controller is evaluated and incrementally tuned and improved in the simulation environment.

The following chapters are structured according to the steps introduced above. Before beginning with the first step, the actual airship drone oblX_proto, for which the flight controller is developed, is introduced in Section 1.3. Chapter 2 then covers the fundamentals that are important for understanding the work. Thereafter, in Chapter 3, the steps 1-3 are addressed and the setup of the dynamic model and the simulation are described. Finally, the steps 4-5 are covered in Chapter 4, in which the implementation of the motor mixer, control allocation and flight controller are presented, following with the evaluation of these components in several tasks and scenarios in Chapter 5. Finally, a conclusion is given in Chapter 6 and the results of the thesis are discussed.

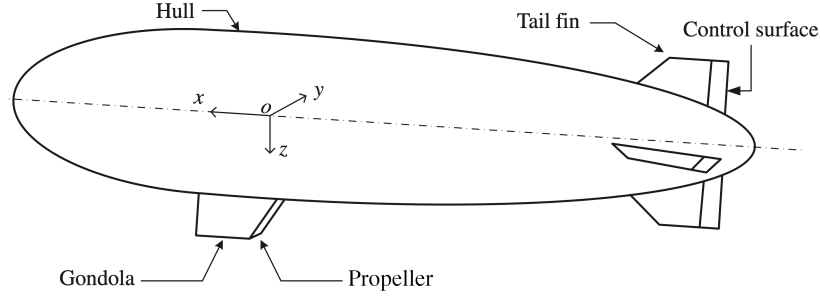
1.2 Related Work

Far-reaching improvements in computational power and hardware efficiency have driven autonomy in robotics. Due to the versatility of UAVs, the trend has also spread from ground-based to aerial robotic systems. As the tasks become more demanding, the requirements on autonomous systems for UAVs have increased. Many tasks require full autonomy where the robot has to engage in unknown environments by perceiving the environment and making decisions solely based on sensor data [ElmokademSavkin21].

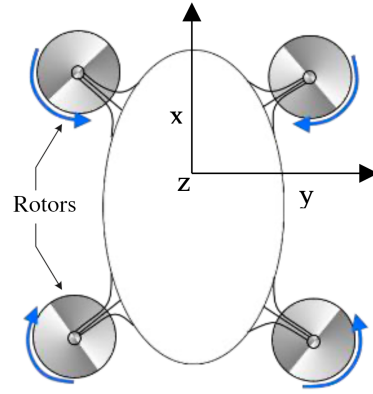
Airship UAVs, however, are often not as well represented in research compared to other types of UAVs. One of the recent approaches towards an autonomous robotic airship system is project AURORA [BuenoEtAl02] which describes a comprehensive framework including the electronics on board, flight control and simulation. One difficulty hereby is the standardization and transferability of algorithms and simulations, since airships often differ in shape, size and propulsion system, making it hard to generalize approaches. Several works, therefore, are aiming to build such frameworks in order to increase the accessibility and provide a generalized approach [PriceEtAl22, GorjupLiarokapis20].

This thesis is specialized in the development of a flight controller with the help of open source software and accessible flight control hardware for an airship with a new type of propulsion system. While conventional airships usually consist of two propellers next to the gondola for generating thrust and rudders for steering, novel propulsion systems try to address the sluggish maneuverability of airships by adding additional actuators. Adding fixed or tiltable thrusters or combining an airship with a multirotor propulsion system, enables more dynamic flight and a higher controllability of the airship. Figure 1.1 illustrates different propulsion systems for airships.

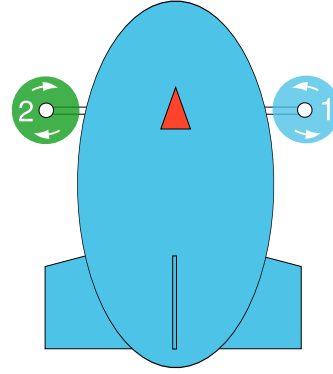
The flight controller for the oblX_proto is built using the PX4 flight stack [PX422]. The PX4 flight stack provides several pre-defined airframes for many types of UAVs. An airframe specifies the number and configuration of all controllable actuators and simplifies the setup of, for example, a motor mixer and implemented controller in PX4. An airframe for an airship with a propulsion system as shown in Figure 1.1 (c) is also available. This propulsion system consists of a tiltable starboard and port thruster, and a tail thruster. Due to the large differences in the propulsion system, the flight controller for the oblX_proto could not be built upon the existing airship airframe of PX4 and a custom airframe had to be implemented.



(a) Conventional airship



(b) Hybrid airship-quadcopter propulsion system



(c) PX4 airship airframe

Figure 1.1: Three airship types with different propulsion systems, where (a) shows the structure of a conventional airship [LiNahonSharf11], (b) an airship with a hybrid airship-quadcopter propulsion system, [MalekSedanMohdHarithuddin18], and (c) the available airship airframe in PX4 with a specific motor configuration containing tiltable rotors [PX422].

1.3 The oblX_proto Airship Drone

The oblX_proto is an airship prototype from roboloon, Stuttgart, with a novel propulsion system consisting of a stern propeller, four rudders and eight side engines that are oriented in a way to create a motion in every direction. The propulsion system allows the airship to maneuver in any direction by actuating specific engines at the same time, leading to a resulting desired movement of the airship. The side engines consist of ducted fans inside the engine housing and are unidirectional. An illustration of the oblX_proto and its components is given in Figure 1.2

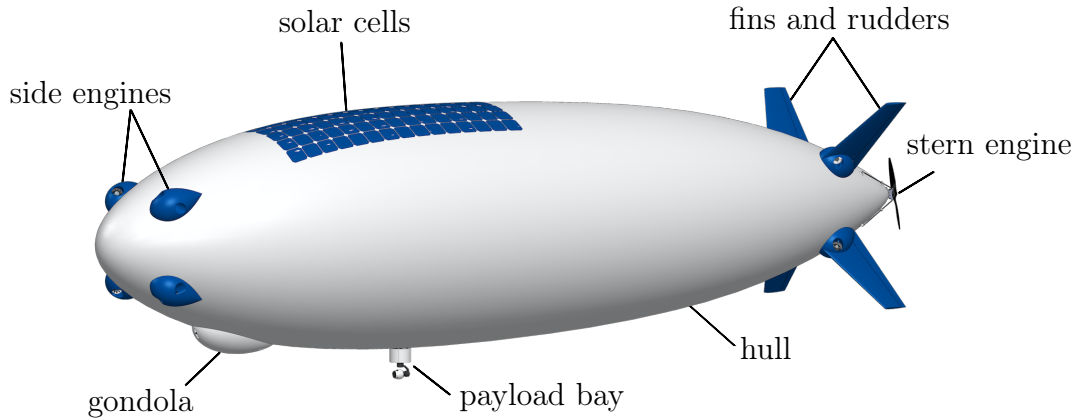


Figure 1.2: CAD model of the oblX_proto with labeled components.

The hull of the airship has a total length of 8 m and a volume of $16,3 \text{ m}^3$, and is filled with helium to generate lift. The hull also has an integrated safety valve to release the helium when necessary. The oblX_proto, furthermore, consists of a gondola that houses the battery, the flight control hardware, a camera and the remaining electrical components. Additional sensors for various use cases can be accommodated on the payload bay, and solar cells on the roof as second energy source provides additional flight time. The airship is designed in a way that the center of gravity (COG) is positioned below the center of volume (COV) and the mass is evenly distributed along the length and width of the airship to ensure a level orientation in hovering state without additional work.

A Cube Orange [Cub22], or also named Pixhawk 2.1, is used as flight control hardware, which in total contains two processing units, three accelerometers, three gyroscopes, two barometers and a compass. The redundancy in the sensors increase the reliability of the measurements and allows for a more accurate state estimation. In addition, an external GPS unit is connected to the Pixhawk. A Herelink radio controller [Her22] is used for the telemetry, which is connected to the Pixhawk over a receiver and allows the monitoring and control of the airship through manual control and mission planing with the integrated ground control station. The Herelink establishes a connection with a range of up to 20 km and enables the transmission of video data in real time.

Chapter 2

Fundamentals

This chapter provides an introduction into the fundamentals that are necessary for the subsequent chapters. It starts with the dynamics of the airship drone, given in Section 2.1. The airship `oblX_proto` is modeled by elaborating the Newton-Euler equations of motion of the system. The `oblX_proto` is an over-actuated system and solving the equations of motion for a desired motion results in infinite possible combinations to actuate the motors. An optimal combination of the motors can be found by running a linear optimization. Section 2.2, therefore, introduces the basic concept of linear optimization and presents a method to solve a stated linear optimization problem.

2.1 Dynamics of the Airship Drone

The airship drone is a UAV that, in its simplest form, consists of an hull filled with a gas less dense than air, a propeller and rudders. The propeller creates a forward thrust while the rudders are used to steer the airship by changing its attitude. Airships in general show great similarities to submarines due to their buoyancy and high inertia. The high inertia in contrast to the relatively low mass is one major difference to other UAV types, which is primarily due to the large volume of the airship that causes additional aerodynamic and fluid-dynamic effects.

Since the dynamics of an airship resembles the dynamics of a submarine, a way to model the dynamics of an airship is to consider it as a vehicle in a moving fluid [Thomasson00]. In a perfect fluid, additional forces are needed to accelerate the airship in contrast to an acceleration in vacuum. This additional work increases the kinetic energy in the outer fluid and appears as an increase in the mass of the vehicle, which is referred to as the added mass. This added mass represents

the additional energy that has to be transferred to the fluid during acceleration [Thomasson00]. The aerodynamics of the airship can be modeled accordingly using on the added mass of the system [LiNahonSharf11].

However, in this thesis, it is assumed that the aerodynamic effects appear in the form of an external force depending on the physical and geometrical properties of the vehicle, and that the mass of the airship is constant. It is also assumed that the weight of the components of the airship and the forces generated by the propulsion system do not affect the shape of the airship and, therefore, the airship is regarded as a rigid body with fixed joints. The dynamic model in the subsequent section is based on the physical and geometrical specifications of the oblX_proto as specified in Section 1.3. The airship is designed such that it is symmetrical to the x-z-plane and the COG is centered below the COV. The actuation of the rudders is also neglected and therefore only the engines are considered for an active manipulation of the position and the orientation of the airship.

2.1.1 Kinematics and Dynamics

The kinematics of the airship is modeled using the Z-Y-X Euler angles convention and two reference frames, as illustrated in Figure 2.1. The fixed inertial frame I is given by the axis x , y and z , with z pointing upwards. For the body reference frame B of the airship, the North East Down (NED) frame convention is applied. The origin of B is positioned at the COG of the vehicle and the frame axes are given by x_B , y_B and z_B , with x_B pointing to the nose of the airship and z_B pointing downwards. In ideal hovering state, z_B is parallel to z .

The reference frame B is given by a sequence of rotations along the z -, an intermediate y' - and an intermediate x'' -axis, being congruent to the resulting x_B -axis. The associated rotation matrix is given by

$${}^I\mathbf{R}^B = \begin{bmatrix} c_\psi c_\theta & -s_\psi c_\phi + c_\psi s_\theta s_\phi & s_\psi s_\theta + c_\psi c_\phi s_\theta \\ s_\psi c_\theta & c_\psi c_\phi + s_\psi s_\theta s_\phi & -c_\psi s_\phi + s_\psi s_\theta c_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix}, \quad (2.1)$$

where $s_{(\cdot)}$ and $c_{(\cdot)}$ denote $\sin(\cdot)$ and $\cos(\cdot)$, respectively [Fossen11].

The vehicle has 6 DOF due to the number and the configuration of the actuators. Since the actuators are fixed to the main body of the airship and the hull of the airship is assumed remain its shape, the vector of general coordinates is given by the cartesian coordinates, with

$$\mathbf{y} = \begin{pmatrix} x_B \\ y_B \\ z_B \\ \phi_B \\ \theta_B \\ \psi_B \end{pmatrix}. \quad (2.2)$$

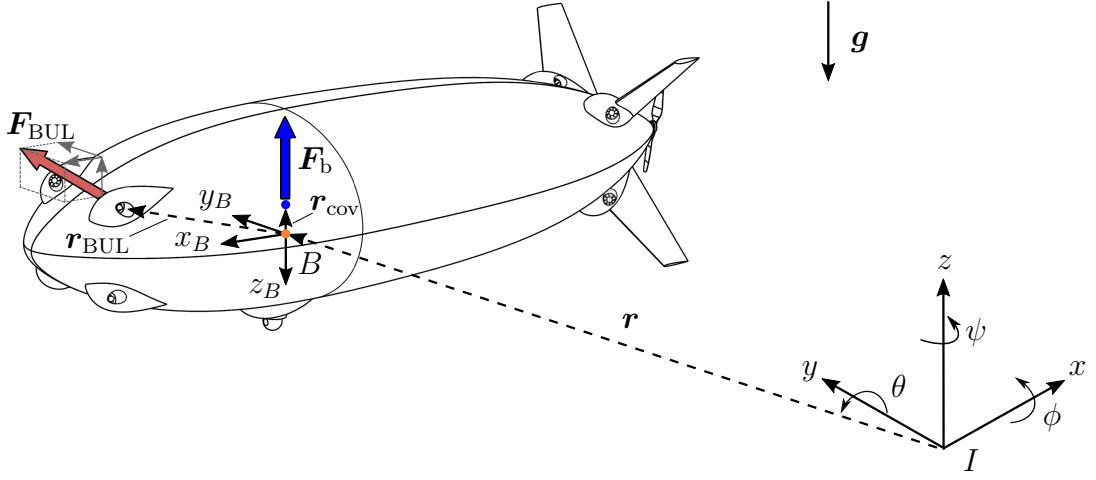


Figure 2.1: Schematic representation of the dynamics of the oblX_proto. The body frame B is positioned at the COG of the airship, represented by the orange dot and being at the global position $\mathbf{r}$, while the inertial frame I is fixed. The airship experiences a buoyancy $\mathbf{F}_B$ at the COV, represented by the blue dot. The thrust generated by a side engine is given as an example with $\mathbf{F}_{\text{BUL}}$ for the bow-up-left (BUL) side engine.

The dynamics of the system in its general form without consideration of the actuators is given by the Newton-Euler equation of motion:

$$\mathbf{M}(\mathbf{y}, t) \ddot{\mathbf{y}} + \mathbf{k}(\mathbf{y}, \dot{\mathbf{y}}, t) = \mathbf{q}(\mathbf{y}, \dot{\mathbf{y}}, t), \quad (2.3)$$

where $\mathbf{M}(\mathbf{y}, t) \in \mathcal{R}^{6 \times 6}$ denotes the mass matrix, $\ddot{\mathbf{y}} \in \mathcal{R}^6$ denotes the second derivative of the vector of general coor $\mathbf{y}$, $\mathbf{k}(\mathbf{y}, \dot{\mathbf{y}}, t) \in \mathcal{R}^6$ denotes the vector of Coriolis and centripetal forces, and $\mathbf{q}(\mathbf{y}, \dot{\mathbf{y}}, t) \in \mathcal{R}^6$ denotes the vector of external forces. Since the airship is assumed to be a rigid body and the body frame B is given at the COG, the mass matrix can be simplified to:

$$\mathbf{M} = \left[\begin{array}{c|c} \sum_i m_i \mathbf{I}_3 & \mathbf{0}_{3 \times 3} \\ \hline \mathbf{0}_{3 \times 3} & \sum_i \mathbf{I}_i^{COG} \end{array} \right], \quad (2.4)$$

where m_i denotes the total mass of the i -th component of the airship, $\mathbf{I}_3 \in \mathcal{R}^{3 \times 3}$ denotes the identity matrix and $\mathbf{I}_i^{COG} \in \mathcal{R}^{3 \times 3}$ denotes the inertia of the i -th component of the airship w.r.t the COG. The vector of Coriolis and centripetal forces can also be simplified to the following expression:

$$\mathbf{k}(\dot{\mathbf{y}}, t) = \begin{pmatrix} \mathbf{0}_3 \\ \boldsymbol{\omega}_B \times (\mathbf{I}^{COG} \boldsymbol{\omega}_B) \end{pmatrix}, \quad (2.5)$$

where the dependency of $\mathbf{k}(\dot{\mathbf{y}}, t)$ on $\mathbf{y}$ disappears due to the rigid body assumption. The body angular velocity vector

$$\boldsymbol{\omega}_B = \begin{pmatrix} \dot{\phi}_B \\ \dot{\theta}_B \\ \dot{\psi}_B \end{pmatrix} = \begin{bmatrix} 1 & 0 & -s_\theta \\ 0 & c_\phi & c_\theta s_\phi \\ 0 & -s_\phi & c_\theta s_\phi \end{bmatrix} \begin{pmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{pmatrix} \quad (2.6)$$

describes the angular velocity vector of the airship given in the airship frame B , and $\mathbf{I}^{COG} = \sum_i \mathbf{I}_i^{COG}$ [Fossen11]. For detailed derivation of $\boldsymbol{\omega}_B$ reference is made to [McGheeBachmannZyda01].

The external forces acting on the vehicle are the gravitational force $\mathbf{F}_g$ of the total mass $m = \sum_i m_i$ acting at the COG, the buoyant force $\mathbf{F}_b$ and an aerodynamical force $\mathbf{F}_a$. The gases inside and outside the hull are assumed to be evenly distributed, hence $\mathbf{F}_b$ is acting at the center of volume (COV). The buoyancy is given by the difference of the density of air and the density of the gas inside the hull with

$$\mathbf{F}_b = V \mathbf{g} (\rho_{\text{air}} - \rho_{\text{gas}}), \quad (2.7)$$

where V denotes the volume of the gas inside the hull and $\rho_{(\cdot)}$ denotes the specific gas constant of the respective gases. In an ideal hovering state, the airship is evenly balanced and horizontally aligned so that z_B is parallel to z . When the airship leaves the horizontal orientation, the buoyancy creates a counter torque depending on the roll angle ϕ_B and pitch angle θ_B . In addition, the airship experiences an aerodynamical force $\mathbf{F}_a$ depending on the relative velocity of the air to the vehicle. The relative velocity of the air consists of the velocity of the vehicle and the wind speed. The term for the aerodynamical force depends on the aerodynamical model used for the vehicle and is generalized into an aerodynamical force $\mathbf{F}_a$ and an induced torque $\boldsymbol{\tau}_a$ for simplicity. The used aerodynamic model

is specified in Section 3.3.2. The resulting term for the vector of external forces is given by:

$$\mathbf{q}(\mathbf{y}, t) = {}^I\mathbf{R}^B \begin{pmatrix} \mathbf{F}_g - \mathbf{F}_b + \mathbf{F}_a \\ -\mathbf{r}_{\text{COV}} \times \mathbf{F}_b + \boldsymbol{\tau}_a \end{pmatrix}, \quad (2.8)$$

where $\mathbf{r}_{\text{COV}}$ denotes the vector from the COG to the COV. Due to the design of this particular airship, $\mathbf{r}_{\text{COV}}$ has only a component in the z_B -direction. Since the airship is assumed to be a rigid body, the dependency on $\dot{\mathbf{y}}$ in $\mathbf{q}(\mathbf{y}, t)$ disappears.

2.1.2 Inverse Kinematics and Inverse Dynamics

The airship consists of nine actuators in total: eight side engines and one stern propeller. The stern propeller creates a force in the positive x_B -direction, whereas each side engine is positioned and oriented to create a force in one of the possible directions of $[\pm 1, \pm 2, \pm 1]^\top$, expressed in the body frame B . A general direction vector of the forces generated by the respective side engines is given by

$$\mathbf{d}_{F_{\text{SE}}} = \begin{pmatrix} \pm 1 \\ \pm 2 \\ \pm 1 \end{pmatrix} \quad \text{and} \quad \bar{\mathbf{d}}_{F_{\text{SE}}} = \begin{pmatrix} \pm 1 \\ \pm 2 \\ \pm 1 \end{pmatrix} \frac{1}{\sqrt{6}}. \quad (2.9)$$

$\bar{\mathbf{d}}_{F_{\text{SE}}}$ denotes the normalization of $\mathbf{d}_{F_{\text{SE}}}$. The respective vectors for all engines are listed in Table 2.1 and are all given in the body frame B .

Table 2.1: TABLE

$\mathbf{d}_{\text{BUL}}$	$\mathbf{d}_{\text{BUR}}$	$\mathbf{d}_{\text{BLR}}$	$\mathbf{d}_{\text{BLL}}$	$\mathbf{d}_{\text{SUL}}$	$\mathbf{d}_{\text{SUR}}$	$\mathbf{d}_{\text{SLR}}$	$\mathbf{d}_{\text{SLL}}$	$\mathbf{d}_{\text{SP}}$
$\begin{pmatrix} 1 \\ 2 \\ -1 \end{pmatrix}$	$\begin{pmatrix} 1 \\ -2 \\ -1 \end{pmatrix}$	$\begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix}$	$\begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}$	$\begin{pmatrix} -1 \\ 2 \\ -1 \end{pmatrix}$	$\begin{pmatrix} -1 \\ -2 \\ -1 \end{pmatrix}$	$\begin{pmatrix} -1 \\ -2 \\ 1 \end{pmatrix}$	$\begin{pmatrix} -1 \\ 2 \\ 1 \end{pmatrix}$	$\begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$

A normalization of the listed vectors is denoted with $\bar{\mathbf{d}}_{(\cdot)}$, respectively. A visualization of the direction of the respective forces is shown in Figure 2.2.

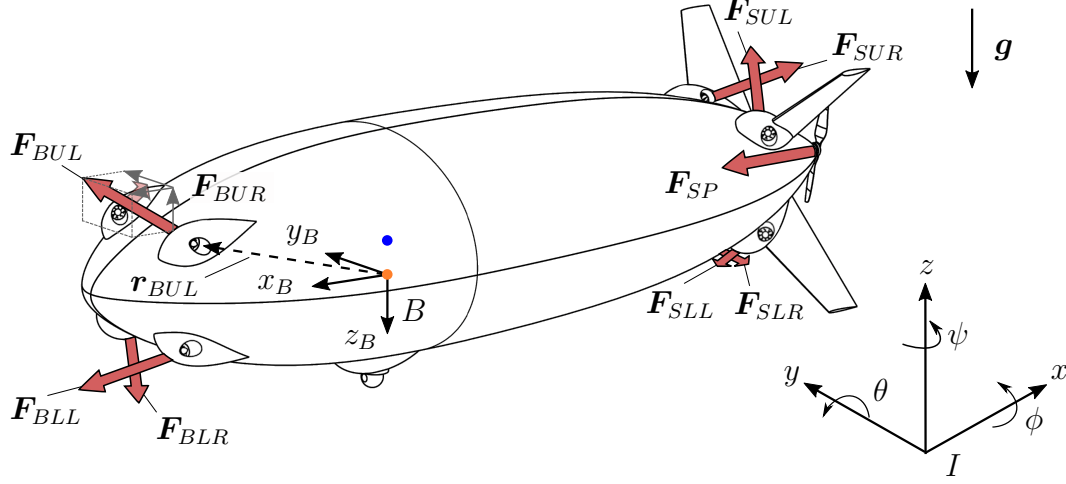


Figure 2.2: Schematic representation of the force vectors generated by the engines. The abbreviations stand for, for example, bow-up-right (BUR), stern-low-left (SLL) and stern propeller (SP). Each engine has its predefined normalized force direction vector $\bar{\mathbf{d}}_{(\cdot)}$.

In order to describe the effect of the motors on the airship when they are actuated, Equation 2.3 must be extended by an additional term for the forces and torques generated by the motors:

$$\mathbf{M}\ddot{\mathbf{y}} + \mathbf{k}(\dot{\mathbf{y}}, t) = \mathbf{q}(\mathbf{y}, t) + \mathbf{B}_{\text{act}}\mathbf{u}_{\text{act}}. \quad (2.10)$$

The matrix $\mathbf{B}_{\text{act}}$ determines the force and the torque that is generated when an actuator command $\mathbf{u}_{\text{act}}$ is applied. $\mathbf{B}_{\text{act}}$ is depending on the engine position, engine orientation and the motor coefficient of the respective engines, whereas $\mathbf{u}_{\text{act}}$ can be described entirely by the angular velocity ω_i of the rotors, with

$$u_{\text{act},i} = \omega_i^2, \quad i = 1, 2, \dots, 9, . \quad (2.11)$$

The force generated by a rotating engine is perpendicular to the direction of rotation and can be described with a general model of a motor, given by:

$$F_i = k_{F,i}\omega_i^2, \quad i = 1, 2, \dots, 9, \quad (2.12)$$

where $k_{F,i}$ denotes the respective engine- and rotor-specific coefficient. The expression for $\mathbf{u}_{\text{act}}$ in Equation 2.11 is derived from the motor model of Equation

2.12 [PowersMellingerKumar15]. A rotating engine creates an additional moment along the axis of rotation, which can be neglected due to the comparatively large inertia of the airship and its inherent stability due to the low center of gravity and its symmetrical mass distribution. The resulting forces and torques created by the engines are given by:

$$\begin{pmatrix} \mathbf{F}_{eng,res} \\ \boldsymbol{\tau}_{eng,res} \end{pmatrix} = \mathbf{B}_{act} \mathbf{u}_{act} = \begin{bmatrix} k_{F_{SE}} \bar{\mathbf{d}}_{F_{BUL}} & \cdots & k_{F_{SP}} \bar{\mathbf{d}}_{F_{SP}} \\ \mathbf{r}_{BUL} \times k_{F_{SE}} \bar{\mathbf{d}}_{F_{BUL}} & \cdots & \mathbf{r}_{SP} \times k_{F_{SP}} \bar{\mathbf{d}}_{F_{SP}} \end{bmatrix} \begin{pmatrix} \omega_1^2 \\ \vdots \\ \omega_9^2 \end{pmatrix}, \quad (2.13)$$

where $\mathbf{r}_{(\cdot)}$ denotes the position vector of an engine given in the body frame B . The first eight engines are defined as the side engines and therefore the coefficient $k_{F,i}$ for $i = 1, \dots, 8$ is given by $k_{F_{SE}}$. Likewise for the ninth engine, which is the stern engine, $k_{F,i}$ for $i = 9$ is given by $k_{F_{SP}}$.

To follow a desired trajectory $\mathbf{r}_d$, the desired state $\mathbf{y}_d$ at each time step can be computed with the inverse kinematics:

$$\mathbf{y}_d = \mathbf{r}^{-1}(\mathbf{r}_d). \quad (2.14)$$

With the inverse dynamics, the desired control command $\mathbf{u}_d$ can be computed at each time step for the given $\mathbf{y}_d$, $\dot{\mathbf{y}}_d$ and $\ddot{\mathbf{y}}_d$, substituted into Equation 2.10:

$$\mathbf{u}_{act,d} = \mathbf{B}_{act}^{-1} [\mathbf{M} \ddot{\mathbf{y}}_d + \mathbf{k}(\dot{\mathbf{y}}_d, t) + \mathbf{q}(\mathbf{y}_d, t)] \quad (2.15)$$

Since the system is under-determined, there are multiple possible outcomes for $\mathbf{u}_{act,d}$ at each time step t , given a desired state $\mathbf{y}_d$, $\dot{\mathbf{y}}_d$ and $\ddot{\mathbf{y}}_d$. Equation 2.15 only provides a direct mapping of a desired state to a desired control command for a feed forward control and does not take into account specific conditions. Additional constraints of the propulsion system like the unidirectionality of the engines are not implemented and therefore not considered. The control command has to be allocated to the actuators with a logic that considers the specific constraints. The term for the desired control command in the case of a feed forward control can be given analogously to Equation 2.15 with

$$\begin{aligned} \mathbf{u}_{act,d} &= \mathbf{B}_{alloc}(\mathbf{y}, \dot{\mathbf{y}}, t) \mathbf{u}_d, \\ \mathbf{u}_d &= \mathbf{B}_{alloc}(\mathbf{y}, \dot{\mathbf{y}}, t)^{-1} \mathbf{B}_{act}^{-1} [\mathbf{M} \ddot{\mathbf{y}}_d + \mathbf{k}(\dot{\mathbf{y}}_d, t) + \mathbf{q}(\mathbf{y}_d, t)]. \end{aligned} \quad (2.16)$$

$\mathbf{B}_{\text{alloc}}(\mathbf{y}, \dot{\mathbf{y}}, t)$ represents the allocation matrix and is depending on the dynamics and the constraints of the system. With $\mathbf{B}_{\text{alloc}}(\mathbf{y}, \dot{\mathbf{y}}, t)$, it is possible increase the abstraction level of the control variables from the control of each individual actuator with $\mathbf{u}_{\text{act}}$ to the control of, for instance, desired linear and angular accelerations, or desired forces and torques with $\mathbf{u}_{\text{d}}$. However, to determine an optimal allocation of a desired control command to the actuators, $\mathbf{B}_{\text{alloc}}(\mathbf{y}, \dot{\mathbf{y}}, t)$ has to be determined depending on the desired objective using an optimization method, since the system is under-determined. A static allocation $\mathbf{B}_{\text{alloc}}$ is known as a motor mixer and needs to be determined once.

Furthermore, using Equation 2.16 entirely to follow a trajectory $\mathbf{r}_{\text{d}}$ would be trivial if the model of the vehicle would be known exactly and if there are no external disturbances. Since this is not the case in reality, a closed loop feedback control can be used to determine $\mathbf{u}_{\text{d}}$.

2.2 Linear Optimization

Optimization describes a mathematical method that aims to find a minimum or a maximum of a given performance criterion ζ consisting of n parameters $x_1, x_2, \dots, x_n$:

$$\begin{aligned} &\text{minimize } \zeta = f(x_1, x_2, \dots, x_n) \text{ , or} \\ &\text{maximize } \zeta = f(x_1, x_2, \dots, x_n). \end{aligned} \tag{2.17}$$

ζ is synonymously referred to as the objective or cost function and the problem of finding the optimal solution over all possible solutions is referred to as the optimization problem. The approaches to solve the optimization problem can be categorized into analytical, graphical, experimental and numerical methods, where numerical methods represent the most important general approach [AntoniouLu07]. While analytical methods reach their limits with many highly complex problems, numerical methods are able to solve these by iteratively and numerically improving the solution until the optimum or a defined convergence criterion is reached.

The practice of numerical optimization is also known as mathematical programming [AntoniouLu07]. Linear optimization, also referred to as linear programming [Dantzig63], is a branch of mathematical programming in which the objective function ζ is linear and the constraints consist of linear equality and inequality expressions [LuenbergerYe21]. The linear optimization problem can be expressed in the following general form:

$$\begin{aligned} & \text{minimize } \zeta = \mathbf{c}^\top \mathbf{x} \\ & \text{subject to } \mathbf{A}\mathbf{x} = \mathbf{b} \text{ and } \mathbf{x} \geq 0, \end{aligned} \quad (2.18)$$

where $\mathbf{x} \in \mathcal{R}^n$ denotes the decision vector, $\mathbf{c}^\top \in \mathcal{R}^n$ denotes the objective coefficient vector as a row vector, $\mathbf{A} \in \mathcal{R}^{m \times n}$ denotes the constraint matrix, and $\mathbf{b} \in \mathcal{R}^m$ denotes the right-hand side data vector [LuenbergerYe21, Vanderbei20]. The aim of linear program is to find an optimal solution for the data vector $\mathbf{x}$ that minimizes the objective function ζ and satisfies the specified constraints of the Equation 2.18. The linear optimization problem can also be formulated to maximize the objective function ζ .

2.2.1 Basic Feasible Solutions

$\mathbf{A}\mathbf{x} = \mathbf{b}$ represents a linear system of equalities. Depending on the sizes of the dimensions m and n , the system can be under- or over-determined:

- **Under-determined system:** for an under-determined system, the number of variables n is higher than the number of equality constraints m , which leads to an infinite number of solutions for $\mathbf{A}\mathbf{x} = \mathbf{b}$. In this case, additional constraints, like the minimization of the objective function ζ introduced in Equation 2.18, are necessary in order to generate a unique solution out of the infinite possibilities.
- **Over-determined system:** in contrast, for an over-determined system, the number of variables n is lower than the number of equality constraints m . In this case, there is no direct solution to the stated problem. The solution of the optimization problem involves minimizing the error of the system of linear equations, for example by finding an approximation of $\mathbf{x}$ with the least square of the error [BruntonKutz19]:

$$\hat{\mathbf{x}} = \arg \min_x \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2. \quad (2.19)$$

For the linear optimization problem, we assume that $\mathbf{A}$ has full row rank m and full column rank n . Furthermore, we assume that $m < n$ always holds and the system is consequently under-determined. In an under-determined system where $\mathbf{A}$ has m linear independent rows and n linear independent columns, a basic feasible solution is given by

$$\mathbf{B}\mathbf{x}_B = \mathbf{b}, \quad (2.20)$$

where $\mathbf{B} \in \mathcal{R}^{m \times m}$ denotes the basis and is a matrix containing a set of m linear

independent columns of $\mathbf{A}$, and $\mathbf{x}_B \in \mathcal{R}^m$ is a vector containing the elements of $\mathbf{x}$ associated with the columns of $\mathbf{B}$. The solution of Equation 2.20 is therefore given by p non-zero variables x_p with $p \leq m$. Therefore, for the optimal solution to Equation 2.18, only $p \leq m$ of n components of $\mathbf{x}$ have to be non-zero [LuenbergerYe21].

2.2.2 The Primal Simplex Method

The idea of the simplex method is to start at a basic feasible solution and move to the next adjacent feasible solution with an improved objective function. The improvement is done iteratively by changing a basic variable x_m with a non-basic variable $x_j, m+1 \leq j \leq n$, that forms a new basis $\mathbf{B}$ with a lower value to the objective function ζ . The result is a new feasible solution to Equation 2.18 with a modified objective function with a lower value. This procedure is repeated until the optimal solution with the lowest value to the objective function is found. In the following, the simplex method is described based on [LuenbergerYe21].

As introduced in the former Section, the constraint matrix $\mathbf{A} \in \mathcal{R}^{m \times n}$ is partitioned into a basis $\mathbf{B} \in \mathcal{R}^{m \times m}$ consisting of a set of m arbitrary columns of $\mathbf{A}$, and matrix $\mathbf{D} \in \mathcal{R}^{m \times (n-m)}$ consisting of the remaining $n-m$ columns of $\mathbf{A}$, with:

$$\begin{aligned} \mathbf{A} &= \left[\begin{array}{c|c} \mathbf{B} & \mathbf{D} \end{array} \right], \\ \mathbf{x} &= \begin{pmatrix} \mathbf{x}_B \\ \mathbf{x}_D \end{pmatrix}, \mathbf{c}^\top = \begin{pmatrix} \mathbf{c}_B^\top & \mathbf{c}_D^\top \end{pmatrix}, \\ \mathbf{B}\mathbf{x}_B + \mathbf{D}\mathbf{x}_D &= \mathbf{b}. \end{aligned} \tag{2.21}$$

Given a basic feasible solution

$$\mathbf{x}_B = \bar{\mathbf{a}}_0 := \mathbf{B}^{-1}\mathbf{b} \geq 0 \quad \text{and} \quad \mathbf{x}_D = \mathbf{0}, \tag{2.22}$$

the corresponding value of the general objective function can be set as

$$\zeta = \mathbf{c}_B^\top \mathbf{x}_B + \mathbf{c}_D^\top \mathbf{x}_D, \tag{2.23}$$

where

$$\zeta_0 = \mathbf{c}_B^\top \mathbf{B}^{-1}\mathbf{b} \tag{2.24}$$

represents the value to the basic solution. Through Equations 2.21, 2.23 and 2.24, the objective function can then be expressed by:

$$\begin{aligned}
 \zeta &= \mathbf{c}_B^\top (\mathbf{B}^{-1} \mathbf{b} - \mathbf{B}^{-1} \mathbf{D} \mathbf{x}_D) + \mathbf{c}_D^\top \mathbf{x}_D \\
 &= \mathbf{c}_B^\top \mathbf{B}^{-1} \mathbf{b} + (\mathbf{c}_D^\top - \mathbf{c}_B^\top \mathbf{B}^{-1} \mathbf{D}) \mathbf{x}_D \\
 &= \zeta_0 + (\mathbf{c}_D^\top - \mathbf{y}^\top \mathbf{D}) \mathbf{x}_D \\
 &= \zeta_0 + \mathbf{r}_D^\top \mathbf{x}_D,
 \end{aligned} \tag{2.25}$$

where $\mathbf{y}^\top$ denotes the simplex multiplier and $\mathbf{r}_D^\top$ denotes the relative cost vector. Equation 2.25 represents the value of the objective function to any feasible solution. This leads to the following expression for the objective function:

$$\zeta = \zeta_0 + \sum_{j=m+1}^n r_j x_j. \tag{2.26}$$

By adjusting the non-basic variables x_j , a statement can be made as to whether there is an advantage in changing the basic solution. If r_j has a negative value for some j , increasing x_j from zero to a positive value would decrease the total cost and thus lead to a better solution. Therefore, by replacing the variable x_j with a worse performing variable x_m from the basis, a new basis B is set which leads to an overall better basic solution. The process of setting up the basis B , computing the objective ζ , and examining a better performing variable x_j for the new basis, represents one iteration of the simplex method. The concrete steps of the simplex method are performed as follows [LuenbergerYe21]:

1. Given the current basis $\mathbf{B}$, compute the current solution $\mathbf{x}_B = \bar{\mathbf{a}}_0 = \mathbf{B}^{-1} \mathbf{b}$
2. Compute the current simplex multiplier $\mathbf{y}^\top = \mathbf{c}_B^\top \mathbf{B}^{-1}$ and the relative cost coefficients $\mathbf{r}_D^\top = \mathbf{c}_D^\top - \mathbf{y}^\top \mathbf{D}$. If $\mathbf{r}_D \geq \mathbf{0}$, stop since the solution is optimal.
3. Select the most negative cost coefficient $r_e, m+1 \leq e \leq n$, and compute $\bar{\mathbf{a}}_e = \mathbf{B}^{-1} \mathbf{a}_e$, where vector $\mathbf{a}_e$ corresponds to the e -th column of $\mathbf{D}$ and represents the candidate to enter the new basis $\mathbf{B}$.
4. If $\bar{\mathbf{a}}_e \leq \mathbf{0}$, stop since the problem is unbounded and cannot be solved. Otherwise, compute $\bar{\mathbf{a}}_0 / \bar{\mathbf{a}}_e$ for $\bar{\mathbf{a}}_e > \mathbf{0}$ element-wise. Determine the element $o \leq m+1$ of $\bar{\mathbf{a}}_0 / \bar{\mathbf{a}}_e$ with the lowest value. The column vector $\mathbf{a}_o$ of $\mathbf{B}$ represents the candidate to leave the basis and is then replaced with the column vector $\mathbf{a}_e$ of $\mathbf{D}$.
5. Update $\mathbf{B}$ and repeat the procedure with the new feasible solution $\bar{\mathbf{a}}_0 := \mathbf{B}^{-1} \mathbf{b}$ that yields to a lower value of ζ until $\mathbf{r}_D \geq \mathbf{0}$.

For an exemplary calculation for finding an optimum using the simplex procedure above, reference is made to [LuenbergerYe21], pages 85-86.

Chapter 3

Software and Simulation Environment

3.1 Flight Stack

Autonomous systems rely on well-integrated software frameworks that ensure the functionality of all modules needed for autonomous flight. Developing a fully-featured UAV software platform including the communication pipeline between several physical systems involves high development costs and expertise in several domains of software engineering. Using an open-source flight stack reduces the development time for flight controllers and comes with several advantages in terms of reliability of the software and feature richness. Open-source flight stacks are usually maintained by a large community of developers and enable researchers to build upon existing research [EbeidEtAl18]. Compared to closed-source flight controllers, there is also a broader hardware support.

A general overview of open-source flight stacks is given in [EbeidEtAl18]. The number of available projects varies due to rapid developments in the field of UAV software. Some of these projects are forks of other existing or even inactive projects, like Betaflight[Boris22] and Cleanflight [Cle22], which differ in their functionalities or focus on specific applications that, for instance, require high performance or support for a specific operating system. Meanwhile, projects like PX4 [PX422], ArduPilot [Ard22], and Paparazzi [Pap22] have been built with a native support for their respective open-source hardware platforms.

The flight controller hardware used for the oblX_proto is built on the Pixhawk platform series [Pix22]. The Cube Orange [Cub22], which is synonymously named The Pixhawk 2.1, is an advancement of the original Pixhawk design. It has a native compatibility with the PX4 and the ArduPilot flight stacks. Both flight

stacks are relatively mature and share a similar middleware. Both also support a variety of air frames, applications and flight modes, autonomous flight, navigation, a ground control station (GCS) and more. In a further assessment of both flight stacks, the PX4 flight stack is preferred for the further development of the flight controller. Main determining factor for the PX4 is the fact that it is more research oriented and it simplifies the implementation of custom air frames and unique setups of a propulsion system. Another factor that should not be neglected is the licensing of the PX4, which has a significant advantage over the licensing model of ArduPilot, especially for commercial use and preservation of own intellectual property.

The PX4 flight stack is core part of a broader UAV platform that includes the QGroundControl (QGC) GCS [QGC22], the Pixhawk hardware platform and the MAVSDK library collection [Mav22] that enables the communication within the software and hardware components. This make it possible to easily integrate a companion computer that, for instance, runs specific tasks or functions of the autopilot like object avoidance [GarcíaMolina19]. An exemplary communication sequence is illustrated Figure 3.1.

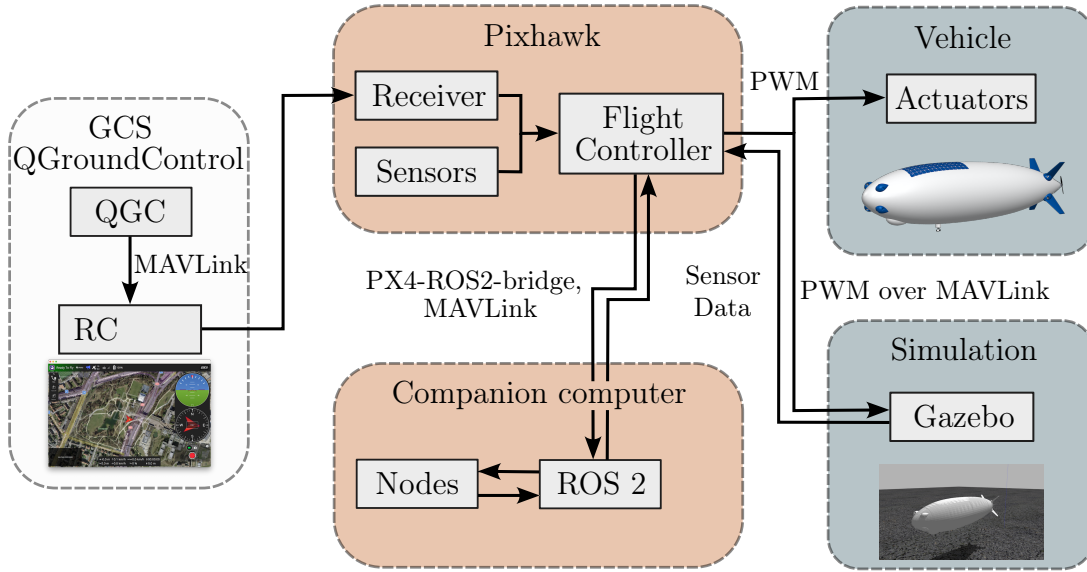


Figure 3.1: Exemplary communication sequence between hardware and software components with PX4.

QGC facilitates the handling of the UAV and visualizes the vehicle and its current state variables on a map. A GCS in general takes on the task of mission planning and monitoring. The vehicle itself communicates with QGC via radio control

unit and a receiver which is connected to the Pixhawk. The Micro Aerial Vehicle Link (MAVLink) communication protocol of the MAVSDK library ensures the communication between the software and hardware modules. It establishes a network of more than 255 concurrent systems like vehicles and QGC instances, which are named MAVLink components. The messages are handled in a hybrid publish-subscribe and point-to-point pattern. While data streams are published to topics and subscribed to, specific command sub-protocols like mission control and parameter control are transmitted point-to-point. Each MAVLink component in the system is supposed to process incoming messages and route messages that are intended for other MAVLink components. This routing of the messages ensures that even components that are not directly connected to the telemetry of the network receive messages, like a companion computer. Micro-services like the heartbeat protocol and the mission protocol implement an additional layer of security for the integrity of the network and the transmitted data [KoubaaEtAl19].

As in the example of Figure 3.1, the MAVLink protocol enables a communication from the QGC through a radio control unit and the flight controller to a companion computer and even to a simulation platform. The PX4 flight stack is, therefore, capable of performing SITL and Hardware-In-The-Loop (HITL). These are particularly useful for testing the flight control algorithm and other software functionalities of the flight controller and autopilot in the simulation and on the hardware before an actual flight.

3.2 Simulation Environment

The simulation of the `oblX_proto` takes place in the Gazebo simulator [KoenigHoward04], a popular open-source 3D dynamics multi-robot simulator that is well-integrated with common frameworks in robotics like the PX4 and Robotic Operating System (ROS) [Sta22]. Gazebo allows the use of different physics engines and provides tools to model robots and create objects and 3D worlds in order to resemble real applications and scenarios. Furthermore, Gazebo enables the modeling of sensors that perceive the 3D environment, like cameras and LiDARs, and other sensors for state estimation, like GPS and IMU. Therefore, it allows for comprehensive testing of algorithms for control, perception, state estimation and machine learning.

The airship drone is modeled in the Simulation Description Format (SDF) which is a robust and extensible format capable of describing robots, objects and environments. The physical properties, kinematic and dynamic attributes, surface properties, sensors and more are described in the SDF. Plugins allow for modeling of sensors and special dynamic properties of the robot and its environment. PX4 provides an interface to Gazebo in form of a wrapper plugin to send sim-

ulated sensor data from Gazebo and receive control commands from PX4 and QGC through MAVLink. This enables a SITL and a HITL of the whole flight controller software and hardware. Figure 3.3 illustrates the communication in the SITL in PX4 with Gazebo and QGC.

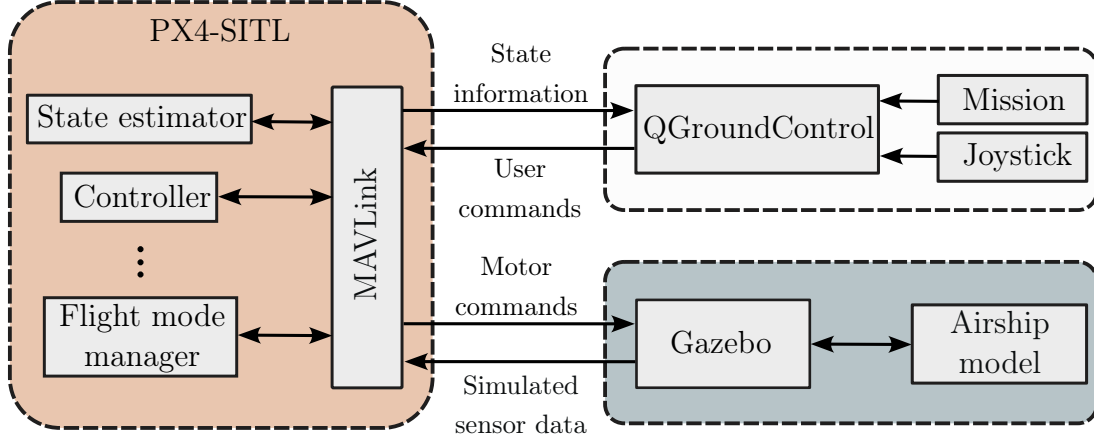


Figure 3.2: TEXT

The SITL enables the utilization and testing of all functionalities of the flight controller software. Therefore, the SITL is set up in this thesis to ensure a comprehensive testing of all modules before an actual flight test. The `oblX_proto` is modeled and simulated in an empty world with a ground in Gazebo, as shown in Figure Figure 3.3. The state information of the `oblX_proto` is given in QGC so that it resembles a real operation. Gazebo provides sensor data from simulated sensors that are available on the Cube Orange. The sensor data is processed by PX4 to estimate the state with an integrated Extended Kalman Filter [ThrunBurgardFox05] of the PX4 library. Commands for launching, landing and flying to a certain point are set in QGC or the PX4 shell and submitted to Gazebo.

When setting a target, like a waypoint in QGC, the flight controller computes the control command, which are forwarded to the simulation through the mavlink wrapper plugin. This control command is then assigned to the engines in the simulation and triggers a motor model plugin that generates a corresponding force. The control command can either come from the flight control algorithm or, alternatively, from a real or virtual joystick. The transferability of the observations and tests from the simulation to the real airship drone, however, depend on the accuracy of the physical model of the airship, the model of the motors, and the aerodynamic model. The next section describes the setup and implementation of the respective models in the simulation.

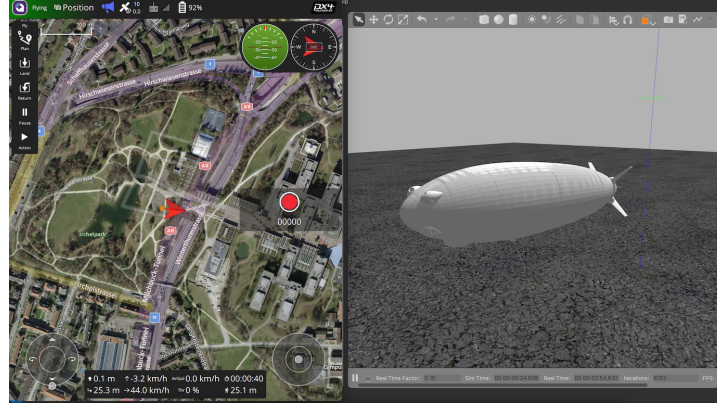


Figure 3.3: PX4-SITL. Left: the QGroundControl GCS, where the vehicle can be controlled and monitored and real scenarios can be emulated in the simulation. Right: the simulation of the oblX_proto in an empty world in Gazebo.

3.3 Airship Model

The validity of the simulation and thus the portability of the flight controller highly rely on the accuracy of the model of the airship drone. In a previous work, a similar smaller-scaled airship drone, the oblX_mini, was modeled and simulated in Gazebo. The oblX_mini is about one fourth of the length of the oblX_proto and its propulsion system is designed for indoor use. However, the existing model of the oblX_mini cannot be transferred and scaled up to the dimensions of the oblX_proto. Therefore, the simulation model of the oblX_proto had to be modeled mostly from scratch.

For the simulation of the airship, the following properties are considered:

- **Physical properties:** The masses and the inertias of the components of the airship drone have to be determined for the dynamic model. Other physical properties like the position and orientation of the engines and rudders are necessary for designing the control algorithm.
- **aerodynamic properties:** The airship is a comparatively huge UAV and is therefore prone to environmental disturbances. The large surfaces cause a non-negligible aerodynamic drag that has to be modeled.
- **Properties of the motors:** As stated in Equation 2.12, the force generated by an engine is assumed to be proportional to the square of the turning speed of a motor. Furthermore, a motor has a certain response time to convert a control signal into a motion. Both characteristics are crucial for the design or, at least, the fine-tuning of the control algorithm.

3.3.1 Physical Model

The airship drone is modeled as a rigid body system in the SDF with the hull being the base body and the components being attached to it with joints and links. The components include the eight side engines, the stern propeller, the four rudders, the payload bay and the gondola. Each component is fixed to the base body and has a predefined position. The whole system is designed according to the real airship, such that the COG is right below the COV to keep the airship in a horizontal orientation.

A multi-body system in the SDF is set up in a tree structure with a base link and several further links attached to it by joints. Each child link can have several child links and joints attached to it in multiple levels. A basic way to model the airship drone is to set up the hull as the base link and attach each component as a child link to the hull with the respective inertial properties assigned to the child link. Gazebo then computes a total mass and a total inertia by itself. However, since the airship is assumed to be a rigid-body, a simpler approach is taken. The total mass and total inertia of each non-moving part, like the gondola, payload bay, side engine housings and the empennage, is computed at the COG and assigned to the hull as the base link. Each individual rotor of the side engines and the stern propeller are child links attached to the hull separately with a revolute joint to enable a rotation in the simulation. The rotors generate a force in the direction of the axis of rotation using a motor model plugin that is implemented as described in Section 3.3.3. Therefore, it is necessary to set each rotor as a child link and define its position and axis orientation in order to enable a rotation in the simulation and specify the point and direction where the thrust is generated.

To determine the geometrical properties and compute the inertial properties of the airship, an existing CAD model of the oblX_proto in SolidWorks [Das22] is taken as a basis and is modified. Since the components of the airship have a relevant size therefore have a non-negligible inertia, modeling these components as mass points would introduce inaccuracies to the resulting inertial tensor and the airship model. For that reason, previously determined surrogate masses and cogs are assigned to the components and the components are positioned in the CAD model accordingly. The total mass and inertia of groups of components are then computed by SolidWorks w.r.t. the COG of the airship. The SolidWorks to URDF Exporter add-in is used to set up the tree structure and export to a URDF file, which is converted to an SDF file by Gazebo.

3.3.2 Aerodynamic Model

Large and long vehicles like an airship or a submarine experience a mixed aerodynamic force that can be described with a drag component and a friction component [HoughtonEtAl13]. While the friction component is proportional to the velocity and is caused by the friction of the air on the surface of the moving object, the air drag is proportional to the square of the velocity and resembles a pressure on a surface. This air drag generates both a drag and a lift and can be modeled with a drag and a lift coefficient, respectively. The modeling of the friction and the air drag, nonetheless, both require an identification of several modeling parameters depending on the shape and size of the airship. More detailed approaches consider further fluid and thermodynamic effects [Drela14].

One way to model the air drag, as mentioned before in Section 2.1, is by computing a mass matrix with the added masses representing the energy that has to be transferred to the fluid during the acceleration of the airship. The added mass matrix is then used to model the air drag of the airship [LiNahonSharf11]. However, this approach requires the identification of several additional modeling parameters. A less complex approach is taken in [PriceEtAl22], where the airship is discretized into several components, with simpler shapes representing the airship, and where each component causes an aerodynamic lift or drag. The resulting force is then summed over all components. This approach requires fewer modeling parameters that can be determined in test flights, thereby approximating the actual aerodynamic behavior of the airship. Due to the current development phase of the airship, these parameters cannot be determined in the scope of this thesis. Therefore, the aerodynamic drag is modeled as a linear damping depending on the velocity and angular velocity of the vehicle:

$$\begin{aligned} \mathbf{F}_d &= c_{d,v} \mathbf{v}_B \\ \mathbf{M}_d &= c_{d,\omega} \boldsymbol{\omega}_B, \end{aligned} \tag{3.1}$$

where $c_{d,(.)}$ denote the respective damping factors for linear or angular damping. The damping factor for the generated force $c_{d,v}$ is determined based on the maximum speed the airship can reach when there is no wind. For $c_{d,\omega}$, however, an estimated value is taken since there is no reference value available and a first flight with the airship was not performed yet. Both are implemented in form of a velocity decay in Gazebo.

3.3.3 Motor Model

The generated thrust of an engine is non-linear to the turning velocity of an engine, depending on the combination of the motor and the attached rotor. Equation 2.12 shows a general relation between these two variables. The engines are

controlled by an electronic speed controller (ESC) that controls the turning velocity of a motor based on the pulse-width modulation (PWM) signal. In most cases the PWM signal can be assumed to be proportional to the turning velocity of an engine. A desired thrust of an engine, hence, can be mapped to a PWM signal. In preliminary studies before, the maximum thrust of a side engine and the stern propeller were determined. Since the maximum PWM value is known and can be assigned to a maximum thrust, the motor coefficients $k_{F,i}$ can be calculated and the thrust generated can be specified as a function of the PWM value, with

$$F_i = k_{F,i} \bar{u}_{\text{PWM},i}^2, \quad i = 1, 2, \dots, 9, \quad (3.2)$$

where $\bar{u}_{\text{PWM}}$ denotes the relative PWM signal w.r.t. the PWM value range.

The latency to reach a certain turning speed is also critical to the dynamical behavior of the vehicle. To take this into account, the two engine types have to be tested for the time needed to turn a control signal into the desired turning velocity. A general linearized model would be

$$\dot{\omega}_i = k_m (\omega_{i,d} - \omega_i), \quad (3.3)$$

where $\omega_{i,d}$ denotes the desired rotor velocity given by the PWM signal and k_m a motor specific latency coefficient [PowersMellingerKumar15]. A latency curve or a value for k_m , however, were not determined in preliminary studies and could not be determined in the scope of this thesis. Therefore, the motor latency model based on the RotorS framework for Gazebo [FurrerEtAl16] is taken with a time constant for an increasing and a time constant for a decreasing turning velocity.

Chapter 4

Flight Controller

The core part of this work is the development of the flight controller for the omnidirectional airship drone. This chapter specifies the design and the implementation of the flight controller based on the vehicle constraints and the constraints set by the flight stack. The flight controller and its modules are developed and continuously tested in the simulation environment, which is set up as described in chapter 3. In the context of UAVs, the term *flight controller* implies the control algorithm, as well as the flight stack and the hardware running the autopilot consisting of the necessary processing units, sensors and I/O ports. In the following chapter, the term flight controller directly corresponds to the control algorithm including the control method and the signal processing pipeline within the flight stack.

Section 4.1 provides an overview of the structure of the flight controller and its individual modules. The sections that follow after then give a more detailed view on the implementation of the modules from lower level to higher level functionalities. Section 4.2 describes how an ideal motor mixer is obtained for the mapping of higher level control commands to the actuation command of individual engines. Since a motor mixer only represents a linear mapping, a dynamic control allocation is implemented as well to achieve a more economic actuation of the engines in autonomous mode, as further elaborated in Section 4.3. Section 4.4 details the structure and the implementation of the PID controller for the flight controller, which itself is divided into a position controller and an attitude controller. The structure of the respective position controller and attitude controller are then described further.

4.1 Flight Controller Architecture

A general overview of the flight controller architecture is given in Fig. 4.1. The flight controller has a modular structure to enable an easy interchangeability and clear interfacing between modules. The modular structure also allows to set up a variety of flight modes for autonomous, semi-autonomous and manual control.

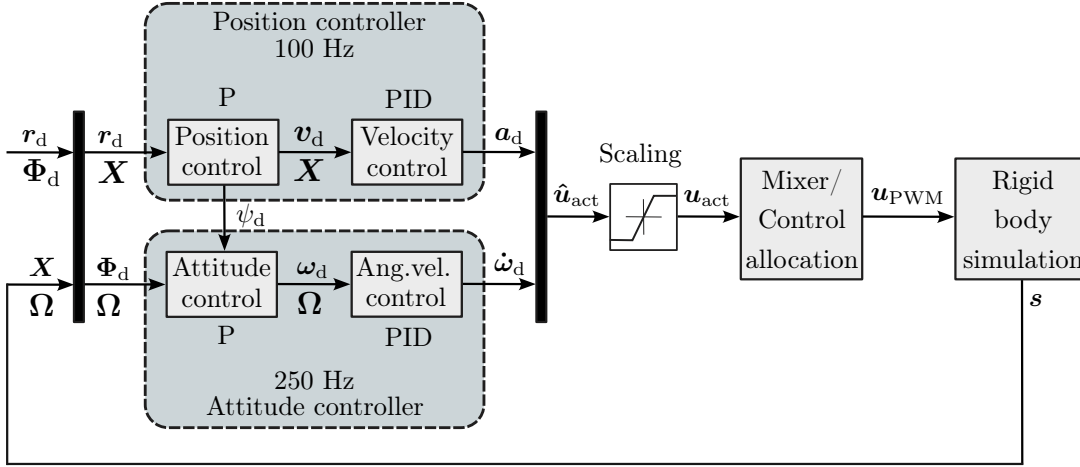


Figure 4.1: The modularized architecture of the flight controller in the simulation. The closed-loop PID feedback controller is split into a position controller and an attitude controller with different cycle times. The control command is scaled down and processed by either the motor mixer or control allocation.

As shown in Figure 4.1, the flight controller receives the information of a desired state $\mathbf{s}_d$ in form of a trajectory or waypoint. The control module then computes a corresponding control command $\mathbf{u}_d$ that reduces the error of the current state $\mathbf{s}$ to the desired state $\mathbf{s}_d$ with $\mathbf{e} = \mathbf{s}_d - \mathbf{s}$. The state vector $\mathbf{s}$ consists of the position $\mathbf{r}$, linear velocity $\mathbf{v}$, linear acceleration $\mathbf{a}$, orientation Φ given in Euler angles, angular velocity ω , and angular acceleration $\dot{\omega}$:

$$\mathbf{s} = \begin{pmatrix} \mathbf{r} \\ \mathbf{v} \\ \mathbf{a} \\ \Phi \\ \omega \\ \dot{\omega} \end{pmatrix} = \begin{pmatrix} \mathbf{X} \\ \Omega \end{pmatrix}, \quad (4.1)$$

where $\mathbf{X} \in \mathcal{R}^9$ consists of the position and its derivatives and $\mathbf{\Omega} \in \mathcal{R}^9$ consists of the attitude and its derivatives. The state variables are computed by a sensor fusion algorithm using the available sensors, like IMU and GPS sensors, and are given in the body frame B .

The control module computes a control command $\hat{\mathbf{u}}_d$. Before $\hat{\mathbf{u}}_d$ is forwarded to the motor mixer or control allocation, it has to be normalized into a range of $[-1, 1]$. Instead of constraining the entries of $\hat{\mathbf{u}}_d$ to a value inside the range, $\hat{\mathbf{u}}_d$ is scaled down to $\mathbf{u}_d$ to maintain the ratio of its entries. $\mathbf{u}_d$ can, for instance, be represented by a desired normalized force and torque in any of the controllable directions, or a desired normalized linear and angular acceleration. The motor mixer then maps the desired control commands $\mathbf{u}_d$ to PWM values $\mathbf{u}_{\text{PWM}}$ which determine the turning velocities of the respective motors. The motor mixer allows to design the controller in a higher abstraction level by introducing an allocation logic that allows to control the vehicle states instead of the individual actuators. In manual control mode, $\mathbf{u}_d$ is directly forwarded to the motor mixer. Since the mapping of a desired control command to the turning velocity of an engine is not always linear, a control allocation module is used in autonomous mode instead of a static motor mixer. The control allocation computes the ideal allocation of $\mathbf{u}_d$ to the individual engines, which allows to minimize the overall actuation effort in any situation compared to the motor mixer.

4.2 Motor Mixer

The system is under-determined due to the number of the actuators being higher than the DOF. Therefore, there are infinite possibilities to map a specific vehicle control command $\mathbf{u}_d$ to an actuator control command $\mathbf{u}_{\text{act}}$ and thus to achieve a desired movement of the vehicle. To allow for a more effective control and overall more efficient flight, the motor mixer is designed to minimize the overall actuation effort.

As shown in Figure 4.2, the motor mixer translates a desired manual control command into a motion of the vehicle. The output of the motor mixer is normalized in the range of $[0, 1]$, where 1 refers to the maximum turning velocity that can be reached by an engine. In this case, the non-dimensional input $\mathbf{u}_d$ can be interpreted as a desired linear or angular acceleration when a manual control input is given. The relation between $\mathbf{u}_d$ and $\mathbf{u}_{\text{act}}$ is then given by

$$\mathbf{u}_{\text{act}} = \mathbf{B}_{\text{mix}} \mathbf{u}_d, \quad (4.2)$$

where $\mathbf{B}_{\text{mix}}$ denotes the mixer matrix. $\mathbf{B}_{\text{mix}}$ is determined by considering the

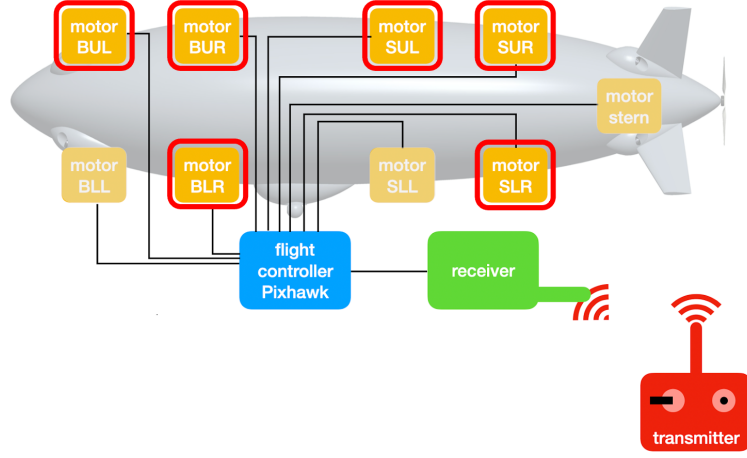


Figure 4.2: Illustration of the functionality of the motor mixer. For example, when the left joystick is positioned to the left, the motor mixer drives specific motors to produce a translation to the left.

dynamics of the airship and the engines modeled with the Newton-Euler equations of motion as stated in Equation 2.10. The mixer represents a static allocation and therefore has to be designed as generally as possible since it follows a fixed logic. A common scenario, for example, involves an operation at small angular velocities, as it would be the case if the airship were being piloted manually. To cover such a general scenario, the following assumptions are made in order to simplify and generalize the motor mixer:

- $\omega_B \approx \mathbf{0} \rightarrow \mathbf{k}(\dot{\mathbf{y}}, t) = \mathbf{0}$: because of the high inertia of the airship and the way the airship is operated, the angular velocities are assumed to be small enough to be negligible. Therefore, the vector of Coriolis and centripetal forces $\mathbf{k}(\dot{\mathbf{y}}, t)$ can be neglected.
- $\theta \approx 0, \psi \approx 0; \dot{\mathbf{y}}_0 \approx \mathbf{0} \rightarrow \mathbf{F}_a = \mathbf{0}, \boldsymbol{\tau}_a = \mathbf{0}$: the lower COG acts like a pendulum w.r.t. the COV, allowing the airship to stabilize itself in a horizontal orientation. The airship is therefore assumed to have a negligible roll and pitch in a stationary state. In addition, for the determination of the mixer, the initial velocities are also assumed to be zero in a stationary state, which eliminates the air resistance according to the implemented aerodynamic model from Section 3.3.2. The vector of external forces $\mathbf{q}(\mathbf{y}, t)$, stated in Equation 2.8, therefore only consist of the aerostatic components.
- $\mathbf{F}_g \approx \mathbf{F}_b \rightarrow \mathbf{q}(\mathbf{y}, t) = \mathbf{0}$: the airship is designed such that under standard conditions the buoyancy is equal to the force of gravity and the COG is

positioned below the COV.. Due to this reason and the previous assumption, the vector of external forces $\mathbf{q}(\mathbf{y}, t)$ is assumed to be negligible in the calculation of the motor mixer. However, not including vector of external forces at all in the calculation of the mixer would be a better practice, since the static components of the external forces can be compensated for in a feed forward control and the dynamic components in a feedback control loop.

In conclusion, the motor mixer is determined based on a steady-state scenario without considering $\mathbf{k}(\dot{\mathbf{y}}, t)$ and $\mathbf{q}(\mathbf{y}, t)$. Furthermore, it must be taken into account that the engines are unidirectional.

4.2.1 Linear Optimization Problem

The determination of the mixer matrix is set as a linear optimization problem according to Equation 2.18, where $\mathbf{A}$ is represented by the actuator matrix $\mathbf{B}_{\text{act}}$, $\mathbf{x}$ is represented by the actuator control command $\mathbf{u}_{\text{act}}$, and $\mathbf{b}$ is represented by the output of the system according to the Newton-Euler equation of motion in Equation 2.10 when a desired control command $\mathbf{u}_d$ is applied:

$$\begin{aligned}\mathbf{A} &= \mathbf{B}_{\text{act}} \\ \mathbf{x} &= \mathbf{u}_{\text{act}} \\ \mathbf{b} &= \mathbf{M}\ddot{\mathbf{y}}_d + \cancel{\mathbf{k}(\dot{\mathbf{y}}, t)} - \mathbf{q}(\mathbf{y}, t).\end{aligned}\tag{4.3}$$

This optimization problem in this setup equals Equation 2.15. $\mathbf{k}(\dot{\mathbf{y}}, t)$ and $\mathbf{q}(\mathbf{y}, t)$ are neglected due to the assumptions made in the beginning of this section. By setting a desired acceleration $\ddot{\mathbf{y}}_d$ which results from $\mathbf{u}_d$ and determines the desired output $\mathbf{b}$, the necessary input $\mathbf{x} = \mathbf{u}_{\text{act}}$ is computed in order to minimize the cost function ζ , given by

$$\zeta = \sum_i x_i = \sum_i u_{\text{act},i}.\tag{4.4}$$

ζ can be interpreted as the total actuation effort that is to be minimized. The desired output $\mathbf{b} \in \mathcal{R}^6$, as stated in Equation 4.3, represents a desired force and torque on the vehicle based on $\mathbf{u}_d$, whereas $\mathbf{u}_d$ represents a desired acceleration $\ddot{\mathbf{y}}_d$. Therefore, the formulation of optimization problem stated in Equation 4.3 is modified to

$$\begin{aligned} \mathbf{A}\mathbf{x} &= \mathbf{b} \\ \mathbf{M}^{-1}\mathbf{B}_{\text{act}}\mathbf{u}_{\text{act}} &= \ddot{\mathbf{y}}_{\text{d}}, \end{aligned} \tag{4.5}$$

to ensure that the resulting motor mixer causes a resulting linear and angular acceleration $\ddot{\mathbf{y}}_{\text{d}}$ by applying a desired normalized control command $\mathbf{u}_{\text{d}}$. Compared to Equation 4.3, Equation 4.5 cancels out unwanted deviation moments caused by a non-centered COG and results in an accurate desired motion of the vehicle. In other words: optimizing for a desired force or torque does not necessarily result in a linear or angular acceleration in this particular direction, if the inertia tensor $\mathbf{I}^{COG}$ has non-diagonal entries.

The mixer matrix $\mathbf{B}_{\text{mix}}$ is obtained by setting one element of $\ddot{\mathbf{y}}_{\text{d}} = \mathbf{u}_{\text{d}}$ to +1 or -1 and the remaining elements to 0, solving the linear optimization problem, and repeating this procedure for the remaining elements of $\mathbf{u}_{\text{d}}$. The result the procedure is the actuator control command $\mathbf{u}_{\text{act}}$ needed in order to obtain the desired acceleration $\ddot{\mathbf{y}}_{\text{d}}$. $\mathbf{u}_{\text{act}}$ is then used to define the respective column of the mixer matrix $\mathbf{B}_{\text{mix}}$. An example of the process is given as follows, where the goal is to determine $\mathbf{u}_{\text{act}}$ for a motion in positive x_B -direction.

First, the optimization problem is set up and solved:

$$\mathbf{M}^{-1}\mathbf{B}_{\text{act}}\mathbf{u}_{\text{act}}^{x+} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}. \tag{4.6}$$

$\mathbf{u}_{\text{act}}^{x+}$ denotes the solution of the linear optimization problem for the x_B -component of $\ddot{\mathbf{y}}_{\text{d}} = \mathbf{u}_{\text{d}}$ being +1 and the remaining elements being 0. The solution to

Equation 4.6 is computed with the simplex method and is normalized to

$$\begin{aligned}\bar{\mathbf{u}}_{\text{act}}^{x+} &= \frac{\mathbf{u}_{\text{act}}^{x+}}{\max \mathbf{u}_{\text{act}}^{x+}} \\ \bar{\mathbf{u}}_{\text{act}}^{x+} &= \begin{pmatrix} 0.1294 \\ 0.1294 \\ 0 \\ 0 \\ 0 \\ 0.1294 \\ 0.1294 \\ 1 \end{pmatrix}, \\ \bar{\mathbf{u}}_{\text{act}}^x &= \begin{cases} u_d^x \geq 0, & \bar{\mathbf{u}}_{\text{act}}^{x+} \\ u_d^x < 0, & \bar{\mathbf{u}}_{\text{act}}^{x-} \end{cases}.\end{aligned}\tag{4.7}$$

$\bar{\mathbf{u}}_{\text{act}}^x$ contains the optimal actuation command of the engines for a positive and a negative motion in the x_B -direction. $\bar{\mathbf{u}}_{\text{act}}^{(\cdot)}$ is a piecewise function depending on the sign of the respective element of $\mathbf{u}_d$. Since the engines are unidirectional, $\bar{\mathbf{u}}_{\text{act}}^x$ has also to be determined for a motion the negative x_B -direction. The relationship between all $\bar{\mathbf{u}}_{\text{act}}^{(\cdot)}$ and $\mathbf{B}_{\text{mix}}$ is then given by:

$$\begin{aligned}\mathbf{B}_{\text{mix}} &= \begin{bmatrix} \sqrt{\bar{\mathbf{u}}_{\text{act}}^x} & \sqrt{\bar{\mathbf{u}}_{\text{act}}^y} & \dots & \sqrt{\bar{\mathbf{u}}_{\text{act}}^x} \end{bmatrix} \\ \mathbf{B}_{\text{mix}} &= \begin{bmatrix} \mathbf{b}_x & \mathbf{b}_y & \dots & \mathbf{b}_\psi \end{bmatrix}.\end{aligned}\tag{4.8}$$

$\mathbf{b}_{(\cdot)} = \sqrt{\bar{\mathbf{u}}_{\text{act}}^{(\cdot)}}$ represents the normalized and optimal allocation vector for generating a desired motion in the respective direction. The expression of $\mathbf{b}_{(\cdot)}$ contains a square root of $\bar{\mathbf{u}}_{\text{act}}^{(\cdot)}$ since the mixer output is supposed to represent a PWM value corresponding to the desired turning velocity of an engine, where $u_{\text{act},i} = \omega_i^2$.

Continuing with the example above, the vector $\mathbf{b}_x$ of the first column of mixer matrix $\mathbf{B}_{\text{mix}}$ is hence given by

$$\mathbf{b}_x = \begin{cases} u_d^x \geq 0, & \mathbf{b}_{x+} \\ u_d^x < 0, & \mathbf{b}_{x-} \end{cases}\tag{4.9}$$

with

$$\mathbf{b}_{x+} = \begin{pmatrix} 0.3597 \\ 0.3597 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0.3597 \\ 0.3597 \\ 1 \end{pmatrix}, \quad \mathbf{b}_{x-} = \begin{pmatrix} 0 \\ 0 \\ 0.2792 \\ 0.2792 \\ 1 \\ 1 \\ 0.96 \\ 0.96 \\ 0 \end{pmatrix}. \quad (4.10)$$

In conclusion, $\mathbf{b}_x$ determines which engines have to be actuated in which intensities to obtain a motion in positive or negative x_B -direction. The elements of $\mathbf{b}_{x+}$ and $\mathbf{b}_{x-}$ are the normalized turning velocities of the engines. Regarding the example above, to achieve a full throttle in positive x_B -direction, the engines BUL, BUR, SLR, SLL have to reach a factor of 0.3597 of their maximum turning speeds and engine STERN has to reach its maximum turning speed. The result is then an isolated motion in the positive x_B -direction. A complete overview of all $\mathbf{b}_{(\cdot)}$ and a visualization of the engines that have to be actuated in order to generate the respective motion can be found in Appendix A.1.

The optimization step of the linear optimization problem is solved using the simplex method, which is explained in Section 2.2.2. The script to compute the motor mixer is written in GNU Octave [EatonEtAl20], using the GNU Linear Programming Kit (glpk) [Makhorin01] as solver.

4.2.2 Input Scaling

To prevent an overdriving of the engines, the input to the motor mixer has to be scaled down to a maximum value. A non-normalized desired control command $\hat{\mathbf{u}}_d$ could lead to an actuator command $u_{\text{act},i} > 1$, if $\hat{\mathbf{u}}_d$ has more than one element above 0 (see detailed motor mixer in Appendix A.1). Clipping $\hat{\mathbf{u}}_d$ or $\mathbf{u}_{\text{act}}$ can lead to a undesired motion of the vehicle since the relation of the forces generated by the respective engines does not fit anymore. $\hat{\mathbf{u}}_d$ could, for instance, lead to a necessary actuation of one engine by 120 %. Instead of clipping this particular engine to 100 %, the command for all engines are scaled down proportionally. The scaling happens as follows:

$$\mathbf{u}_d = \frac{\hat{\mathbf{u}}_d}{\sum_i |\hat{u}_{d,i}|}, i = 1, \dots, 9. \quad (4.11)$$

By scaling down $\mathbf{u}_d$ to the sum of 1, the output of the mixer never exceeds the maximum PWM value for an engine.

4.2.3 Software and Hardware Implementation Issues

To integrate the motor mixer into the PX4 flight stack, the motor mixer for the nine engines have to be split into a main mixer and an aux mixer, since the total number of main PWM outputs in the firmware and on the Pixhawk flight controller hardware are limited to eight in total. For that reason, the whole motor mixer for this engine setup cannot be output as an SBUS signal, since the SBUS connector only outputs the main mixer by default, which, on the other hand, affects the hardware implementation of the mixer. Another limitation regarding the motor mixer is conditioned by the size of the motor mixer file. Each side engine is responsible of a translation or rotation in each of the six DOF, while the stern engine only affects two DOF. Therefore, the side engines have to process six control commands, one for each DOF. This increases the size of the mixer file and, therefore, the motor mixer file causes a memory issue with the I/O processing unit of the Pixhawk. This leads to an issue where the eighth main PWM output cannot be processed by the I/O processing unit. The solution is to minimize the size of the main mixer file by implementing the mixing of one side engine in the aux mixer, and implementing the mixing of the stern engine in the main mixer.

4.3 Control Allocation

The proposed mixer in the previous section starts from a series of assumptions that do not always hold in real conditions, especially when considering the complexity of the propulsion system. A static allocation, like it is the case with a motor mixer, would suffice for a manual control of the vehicle, but could cause issues in dynamic flight tasks with an active control of the vehicle. In regards to energy efficiency, a static mixer starts to be inefficient the moment $\mathbf{u}_d$ has more than one entry above 0, since then more than six of the nine engine are being actuated. As elaborated in Section 2.2, a maximum number of six engines in specific combinations suffice to generate a desired translation or rotation in any direction, since the total number of DOF is six and each side engine of the airship generates a thrust with a component in x_B -, y_B and z_B -direction.

A combined feed forward control of the airship in positive x_B - and y_B -direction, for instance, would actuate eight engines in total according to the mixer given in Appendix A.1. In a case like this, most probably some of the engines actuated are partly canceling each other out and are therefore wasting energy and thereby reducing the flight time. By computing an optimal allocation of the control signal at each time step, the total actuation effort can be minimized noticeably. This dynamic allocation would increase the efficiency by limiting the total number of actuated engine to a total of six while also increasing the performance by

preventing the engines to cancel out each other.

In contrast to a static motor mixer, control allocation algorithms are also able to add additional functionalities to the flight controller, that can be implemented as constraints or secondary objectives. For instance, In the case of a motor failure, the control allocation would allocate the control command to other intact actuators to compensate for the failed actuator [MarksWhidborneYamamoto12]. It is also possible to take the specific characteristics of individual motors into account, like a thrust curve, that, in most cases, is neither linear nor quadratic to the turning speed of the motor and is depending on the voltage level of the battery [PodhradskýEtAl13].

The control allocation algorithm implemented in the flight controller is based on the optimization problem set up for the motor mixer as introduced in the previous section. For an evaluation of several optimization methods for control allocation, reference is made to [Bodson02, GrechiCaiti16]. In the section before, the motor mixer has to be generalized, hence several assumptions had to be made to cover a general scenario, where the motor mixer ensures a desired behavior of the vehicle when applying a higher level control command $\mathbf{u}_d$. The control allocation, on the other hand, is able to cover all occurring situations and combinations of the entries of $\mathbf{u}_d$ and therefore does not need to rely on the assumptions made in the previous chapter.

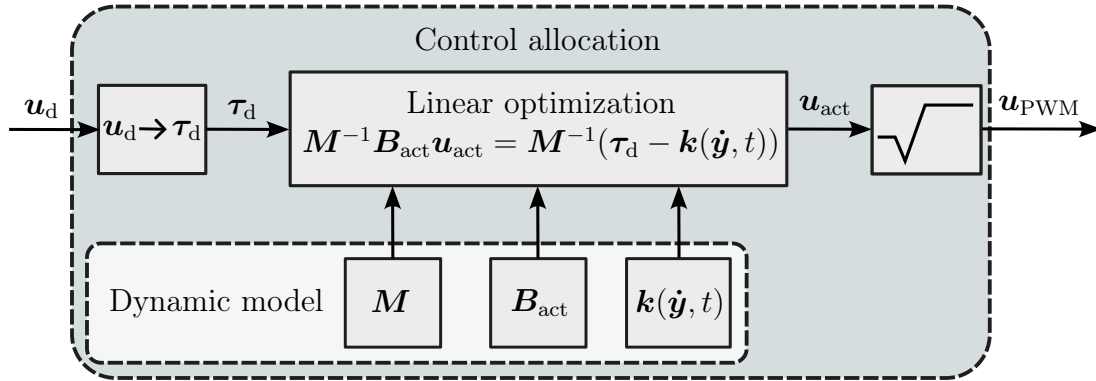


Figure 4.3: Signal flow graph of the control allocation. The control allocation computes an optimal actuation command $\mathbf{u}_{act}$ based on a desired force and torque τ_d and the dynamic model of the vehicle, by stating and solving a linear optimization problem.

A graphical illustration of the control allocation algorithm is given in Figure 4.3. The control allocation module is running in PX4 as a dedicated module and is triggered by the control setpoints coming from the control module. The

optimization problem is modeled and solved with the open-source library glpk in the C++ implementation. The simplex method is used to solve the optimization problem.

4.3.1 Linear Optimization Problem

The implementation of the linear optimization problem is similar to Section 4.2.1, without the assumptions made for the motor mixer. In a first step, the control allocation module takes a normalized and scaled down control command $\mathbf{u}_d \in \mathcal{R}^6$ as input and translates it into desired forces and torques

$$\boldsymbol{\tau}_d = \boldsymbol{\tau}_{\max} \mathbf{u}_d, \quad (4.12)$$

where $\boldsymbol{\tau}_d \in \mathcal{R}^6$ denotes the vector of desired forces and torques, and $\boldsymbol{\tau}_{\max} \in \mathcal{R}^6$ denotes the vector of maximum achievable forces and torques in any given direction. $\boldsymbol{\tau}_{\max}$ is computed once by running the motor mixer, given in Appendix A.1, on full throttle for each direction, which determines the maximum achievable resulting forces and torques in x_B -, y_B - and z_B -direction. In a second step, the optimization problem is set up according to Equation 4.3 with the following modification:

$$\begin{aligned} \mathbf{A} &= \mathbf{B}_{\text{act}}, \\ \mathbf{x} &= \mathbf{u}_{\text{act}}, \\ \mathbf{b} &= \boldsymbol{\tau}_d - \mathbf{k}(\dot{\mathbf{y}}, t). \end{aligned} \quad (4.13)$$

The vector of external forces $\mathbf{q}(\mathbf{y}, t)$ is not considered, since it is compensated for by the control module. The vector of Coriolis and centripetal forces $\mathbf{k}(\dot{\mathbf{y}}, t)$, in contrast, is considered in order to counteract the emerging deviation moments caused by a rotation in combination with a non-centered COG. Analogous to Equation 4.5, the optimization problem is stated as follows:

$$\begin{aligned} \mathbf{A}\mathbf{x} &= \mathbf{b} \\ \mathbf{M}^{-1}\mathbf{B}_{\text{act}}\mathbf{u}_{\text{act}} &= \mathbf{M}^{-1}(\boldsymbol{\tau}_d - \mathbf{k}(\dot{\mathbf{y}}, t)), \end{aligned} \quad (4.14)$$

where the objective function ζ is described by the total actuation effort, as defined before in Equation 4.4. The vector $\mathbf{x} = \mathbf{u}_{\text{act}}$ represents the solution to the linear optimization problem. As with Equation 4.5, the aim of Equation 4.14 is also to generate a desired acceleration when applying a control command $\mathbf{u}_d$, instead of generating a desired force and torque. Each element of $\mathbf{u}_{\text{act}}$ represent the square of the normalized turning velocity $\omega_{\text{norm},i}^2$ of the actuators, respectively. In a final

step, the square root of each element of $\mathbf{u}_{\text{act}}$ is taken and then converted into a PWM signal $\mathbf{u}_{\text{PWM}}$ to be forwarded to the ESCs of the engines, where the lower level velocity control of the engines takes place by a dedicated hardware.

4.3.2 Input Scaling

For the same reason as explained in Section 4.2.2, the input of the control allocator has to be scaled down to prevent an overdriving of the engines and a false ratio of the generated forces by the engines. For the control allocation, both the input $\mathbf{u}_d$ as well as the output $\mathbf{u}_{\text{act}}$ are scaled down according to Equation 4.11 to add more security, in case $u_{\text{act},i} > 1$.

4.4 PID Controller

A PID controller is implemented for the control module of the flight controller. PID controller in general are versatile, easy to comprehend, and robust, if designed and tuned accordingly [GarpingerHägglundÅström14, NambaYamamotoKaneda97]. The PID controller is divided into a position controller and an attitude controller and its controller structure is shown in Figure 4.1. The design of the PID controller is inspired by a standard cascaded control architecture for quadcopter UAVs [PX422, InvernizziEtAl21, PerezEtAl19].

The PID controller receives the information of a desired position $\mathbf{r}_d$ and, if available, a desired attitude Φ_d from a navigator module or a direct user input through QGC. The position controller uses the desired position $\mathbf{r}_d$ and the position state variables $\mathbf{X}$ to minimize the error from the current position $\mathbf{r}$ to the desired position $\mathbf{r}_d$. Likewise, the attitude controller uses the attitude state variables Ω to minimize the error of the current attitude Φ to the desired attitude Φ_d . The position controller, moreover, sets a desired yaw angle ψ_d for the attitude controller. Both the position controller is split into a position and a velocity controller that receives the velocity setpoint from the position controller. In the same way, the attitude controller is split into an attitude and an angular velocity controller which gets the angular velocity setpoint from the attitude controller.

4.4.1 Position Controller

The detailed control architecture of the position controller is given in Figure 4.4. The position controller is divided into a position control and a velocity control to enable a more dynamic control of the vehicle.

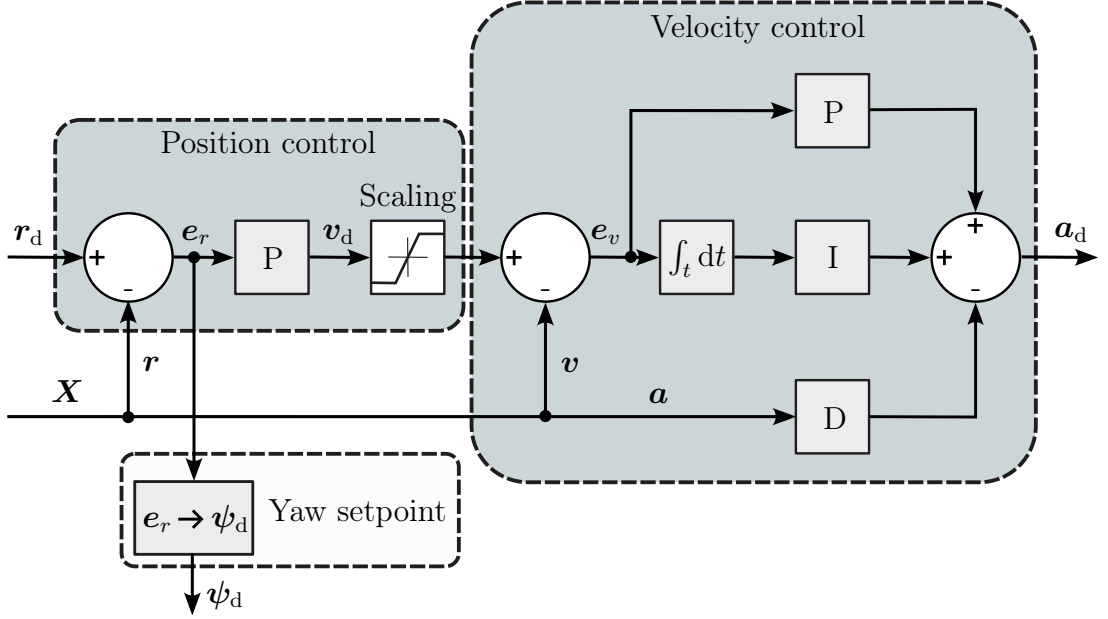


Figure 4.4: Signal flow graph of the cascaded position controller. The position controller is split into a P position control, that computes a velocity setpoint $\mathbf{v}_d$ and a yaw setpoint ψ_d , and a PID velocity control, that computes a desired acceleration based on $\mathbf{v}_d$. $\mathbf{v}_d$ is scaled down, when a lower or an upper bound is exceeded.

The position is controlled by a P controller and the velocity is controlled by a PID controller. The position control sets a velocity setpoint proportional to the position error

$$\begin{aligned} \mathbf{e}_r &= \mathbf{r}_d - \mathbf{r}, \\ \mathbf{v}_d &= \max(\mathbf{k}_{r,p}\mathbf{e}_r; \mathbf{v}_{d,\max}). \end{aligned} \quad (4.15)$$

$\mathbf{k}_{r,p}$ contains the proportional gains for the respective controlled directions. The velocity setpoint $\mathbf{v}_d$ is forwarded to the velocity control module and limited to a maximum velocity setpoint $\mathbf{v}_{d,\max}$ to limit the velocity error. The velocity control module then computes a desired acceleration $\mathbf{a}_d$ accordingly, with

$$\begin{aligned} \mathbf{e}_v &= \mathbf{v}_d - \mathbf{v}, \\ \mathbf{a}_d &= \mathbf{k}_{v,p}\mathbf{e}_v + \mathbf{k}_{v,i} \int_t \mathbf{e}_v dt + \mathbf{k}_{v,d} \frac{d\mathbf{e}_v}{dt}, \end{aligned} \quad (4.16)$$

with $\mathbf{k}_{v,(.)}$ being the respective proportional, integral and derivative gains. $\mathbf{k}_{r,p}$ and $\mathbf{k}_{v,(.)}$ are determined by tuning the gains incrementally, starting with $\mathbf{k}_{r,p}$.

The desired acceleration is passed on to the control allocation to allocate the control commands optimally. A normalization of the velocity control output is not necessary, since the control allocation handles the scaling of the actuator control as described in Section 4.3.2.

The position controller also sets a yaw angle setpoint for the attitude controller. Since the airship is able to maneuver omnidirectionally, it is able to reach any given point in any yaw orientation. However, using the stern engine would be the more efficient way to move along the x_B - y_B -plane. Therefore, the position controller sets a yaw setpoint for the attitude controller to orient the nose of the airship directly towards the desired waypoint with

$$\psi_d = \arccos\left(\frac{e_x}{\|e_r\|_2}\right) \text{sign}(e_y). \quad (4.17)$$

ψ_d represents the yaw angle from the current position to the desired position, and $e_{(\cdot)}$ denotes the respective position error. For small position errors, ψ_d reacts sensitive to changes in e_r . Therefore, for waypoint tracking tasks, ψ_d is computed from the last waypoint to the next waypoint.

4.4.2 Attitude Controller

The attitude controller is set up similarly to the position controller with an attitude control module and an angular velocity control module. Figure 4.5 shows the signal flow of the attitude controller. The attitude control module receives a yaw setpoint ψ_d from the position controller, and a desired attitude Φ_d from a navigator module or a direct user input through QGC. ϕ_d and θ_d are 0 by default in autonomous control, since the airship is supposed to stay in a horizontal position and is able to move in any direction without changing its attitude.

The attitude for the attitude control is given in Euler angles. The desired angular velocity is given by the error from the current attitude to the desired attitude with

$$\begin{aligned} e_\Phi &= \Phi_d - \Phi \\ \omega_d &= \max(k_{\Phi,p} e_\Phi; \omega_{d,\max}). \end{aligned} \quad (4.18)$$

$k_{\Phi,p}$ contains the proportional gains for the respective angles and $\omega_{d,\max}$ denotes the limit for the angular velocity setpoint. ω_d is passed on to the angular velocity controller to compute the desired angular acceleration with

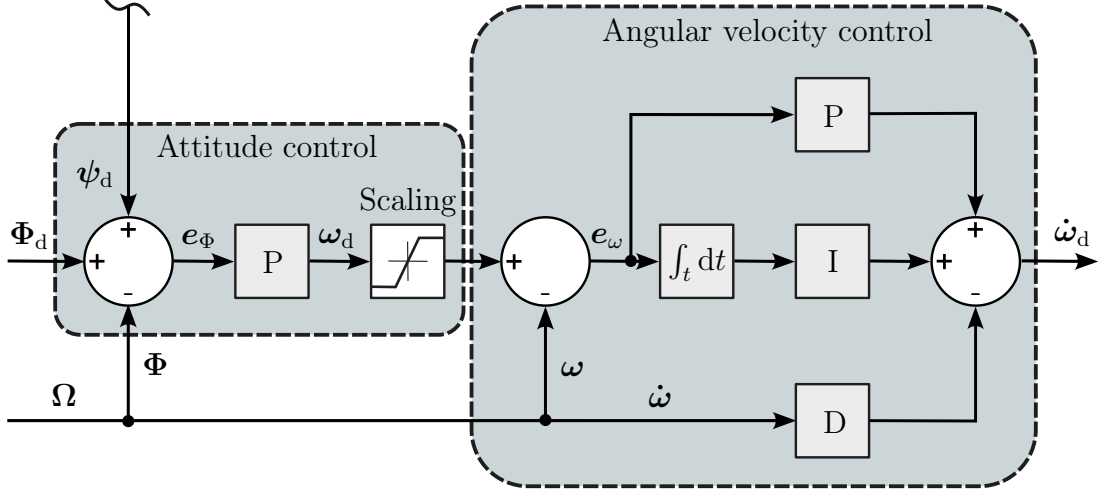


Figure 4.5: Signal flow graph of the cascaded position controller. Similar to the position controller, the attitude controller is split into a P attitude control, that computes an angular velocity setpoint ω_d primarily based on a yaw setpoint ψ_d from the position controller, and a PID angular velocity control, that computes a desired angular acceleration based on ω_d . ω_d is scaled down, when a lower or an upper bound is exceeded.

$$e_\omega = \omega_d - \omega$$

$$\dot{\omega}_d = k_{\omega,p}e_\omega + k_{\omega,i} \int_t e_\omega dt + k_{\omega,d} \frac{de_\omega}{dt}, \quad (4.19)$$

where $k_{\omega,(\cdot)}$ denote the respective proportional, integral and derivative gains. $k_{\Phi,p}$ and $k_{\omega,(\cdot)}$ determined by tuning the gains incrementally, starting with $k_{\Phi,p}$. The desired angular velocity $\dot{\omega}_d$ is passed on to the control allocation module to allocate the control signal the respective engines. The control command for the angular acceleration around the yaw-axis, $\dot{\omega}_\psi$, is additionally weighted higher depending on the yaw error to prioritize the yaw control command:

$$\hat{u}_{d,\psi} = w_\psi e_\psi \dot{\omega}_\psi. \quad (4.20)$$

w_ψ denotes the weighting factor and $u_{d,\psi}$ denotes the control command for the yaw angle. Weighting a control signal of $\hat{u}_d$ does not lead to an overdriving of the engines or a false control of the vehicle. Instead, it weights the respective control signal relative to the other control signals, since $\hat{u}_d$ is scaled down proportionally by the input scaling to a maximum value, as mentioned in Section 4.3.2.

Chapter 5

Evaluation

This chapter discusses the evaluation of the flight controller and the control allocation and motor mixer module. The validation takes place in the simulation environment as described in Chapter 3. The airship is launched in an empty world at the origin of the inertial frame and the tests are carried out either starting from the launching position or 10 m above the launching position. The state information is given by simulated sensors in Gazebo, like a GPS, IMU, barometer and magnetometer sensor, and are fused with a pre-implemented Extended Kalman Filter from the PX4 library.

For the evaluation, several scenarios are examined. The evaluation begins with the validation of the motor mixer and the control allocation in Section 5.1, where both modules are evaluated for their accuracy and effectiveness. The airship drone is omnidirectional, therefore it is examined how accurate an isolated motion in a certain desired direction can take place with both the motor mixer and the control allocation. Since the generation of the desired motion involves the actuation of several motors in specific combinations, it is also examined how effective both modules are in generating it. The effectiveness correlates with the energy-efficiency of the actuation.

In Section 5.2, the performance of the flight controller is evaluated. The ability to follow waypoints and compensate for disturbances are tested in different scenarios. The main scenario is the takeoff and the following of several waypoints during flight. To investigate the behavior against external disturbances, an external force is also applied once during flight and once while maintaining a position.

In the following Sections, all positions and coordinates related to the position are given in the inertial frame I to simplify the representation of position values. In some tests, the airship is 10 m above the ground. Representing the position of the vehicle the inertial frame I is more intuitive since all values on the z_B -axis of the body frame B above the ground are negative. The orientation and coordinates

related to the orientation are, on the other hand, given in the body frame B .

5.1 Validation of the Mixer and Control Allocation

The motor mixer and the control allocation both translate a desired control command $\mathbf{u}_d$ computed from the control algorithm to motor control commands $\mathbf{u}_{act}$ given in PWM values. This mapping provides a less complex interface to the control algorithm and also allows for modularity of components within the flight controller. The flight controller therefore highly relies on the accuracy of the mapping, since a too deviant mapping could lead to an unstable behavior of the vehicle.

In this section, both the accuracy and the effectiveness of the motor mixer and control allocation are evaluated. The motor mixer is initially intended for manual control, but could also be used for automated control in certain circumstances. However, since one of the main goals of this thesis is to develop an energy-efficient controller, a control allocation is used for automated control of the vehicle and its advantages over a motor mixer are shown. The effectiveness provides a metric to evaluate the efficiency of an actuation of the engines. In general, the effectiveness depends on the configuration of the propulsion system and the method to map the control commands in order to obtain a desired motion. Depending on the direction of thrust of the individual engines, the effectiveness of a motion in certain directions is higher compared to that of other directions.

5.1.1 Actuation Accuracy

The accuracy of the motion usually refers to the position error of the actual position to an expected position, when moving the vehicle [ConradShiakolasYih00]. Here, the accuracy refers to the deviation of a desired motion to the actual motion when applying a feed forward control, regarding the velocity. This provides a relative measure and an indication of how accurate the airship is moving in a direction when a feed forward control is applied. For this, the vehicle is launched 10 m above ground and a feed forward control in one specific direction is applied. The deviation is measured for the position coordinates, as well as the attitude coordinates of the vehicle. To determine the metric of how well a desired control command is turned into a motion, the accuracy factor is defined as ratio of the the velocity in the direction that is to be control, to the velocities in the other directions. For example, to measure the accuracy of moving in the x_B -direction, the accuracy factor is calculated with

$$\eta_x = 1 - \frac{\sqrt{v_y^2 + v_z^2}}{v_x}, \quad (5.1)$$

where η_x denotes the actuation accuracy factor in x -direction, and $v_{(\cdot)}$ denotes the velocity components in the respective directions. This measure is more of a measure of how closely the movement produced matches the desired movement. It provides an indication of how much the vehicle deviates in the other directions w.r.t. the desired direction. For a high accuracy value, the vehicle is moving in the desired direction and produces a slight velocity in the other directions. The same calculation is done for the desired angular velocity w.r.t. the other angular velocities. Table 5.1 lists the respective accuracies given w.r.t. the body frame B .

Table 5.1: Actuation accuracies $\eta_{(\cdot)}$.

Feed Forward Control Direction	Accuracy Motor Mixer	Accuracy Control Allocation
η_x	89.91 %	88.87 %
η_y	90.73 %	91.36 %
η_z	98.97 %	98.86 %
η_ϕ	98.13 %	98.14 %
η_θ	98.34 %	99.03 %
η_ψ	91.68 %	96.40 %

The accuracies for the control allocation and the motor mixer are relatively similar. A clear difference is given for the yaw accuracy which is due to the fact that the control allocation considers the dynamics of the vehicle. The control allocation considers the vector of the Coriolis and centrifugal forces in the linear optimization problem, as stated in Equation 4.14. This allows the control allocation to calculate and counteract the deviation when rotating around the yaw axis. For the control allocation as well as the motor mixer, the accuracy factors for the x_B -direction are especially low, since the airship produces an upward pitch of about 10° and an average yaw velocity of about $0.22^\circ/\text{s}$ when moving in the x_B -direction. This might be caused by differences in the model used for the control allocation and the airship model implemented in the simulation.

The motor mixer reaches its limit when $\mathbf{u}_d$ contains more than one entry. For instance, if the feed forward control contains an x_B - and an y_B -component at the same time, the allocation logic of the motor mixer does not hold. To demonstrate it, the motions for using the motor mixer and the control allocation for a constant x_B - and y_B feed forward control are displayed in Figures 5.1 and 5.2.

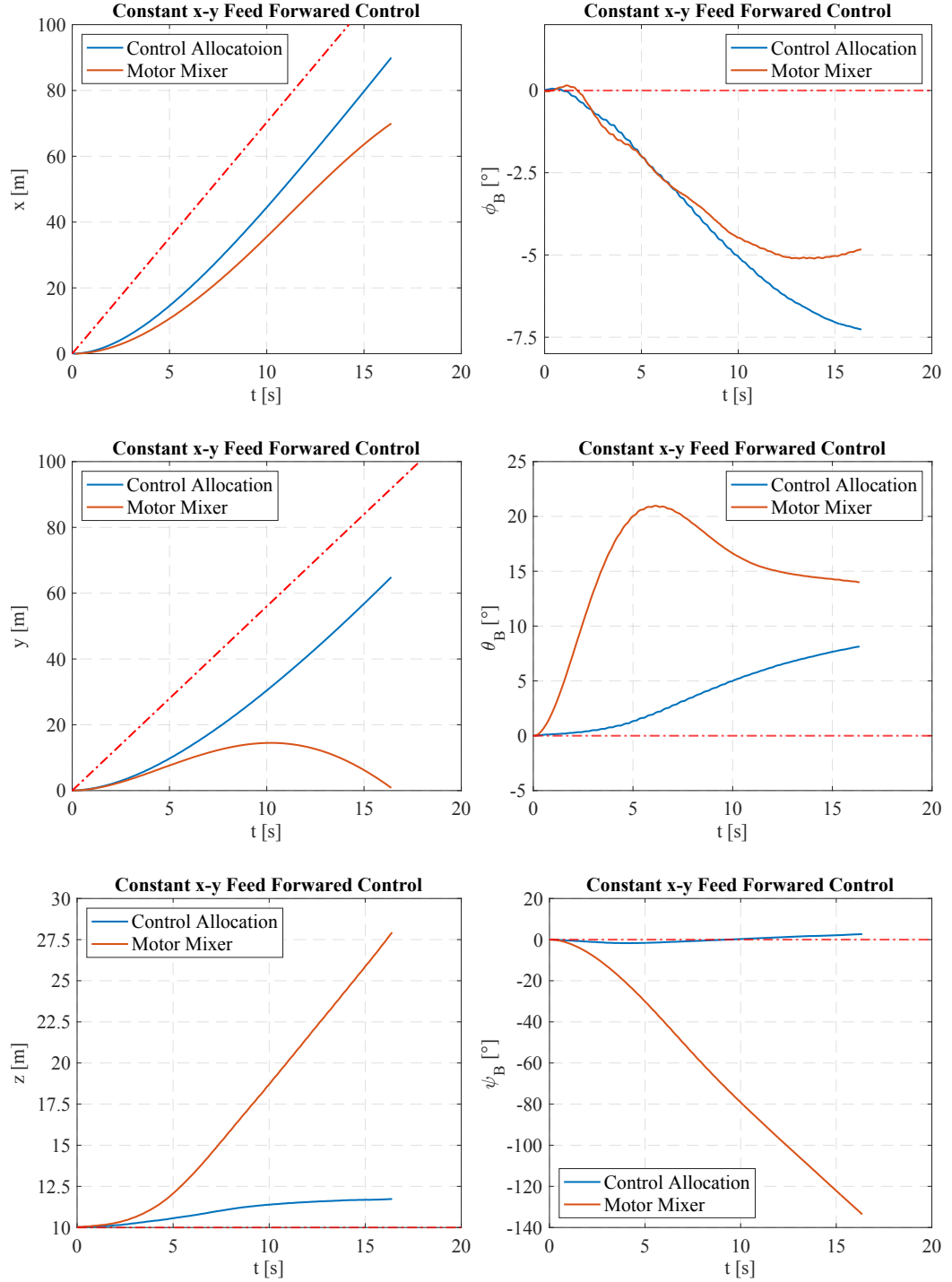


Figure 5.1: Trajectories for a combined constant x_B and y_B feed forward control with the motor mixer and the control allocation given in the body frame B . The red dash-dotted lines indicate the target state.

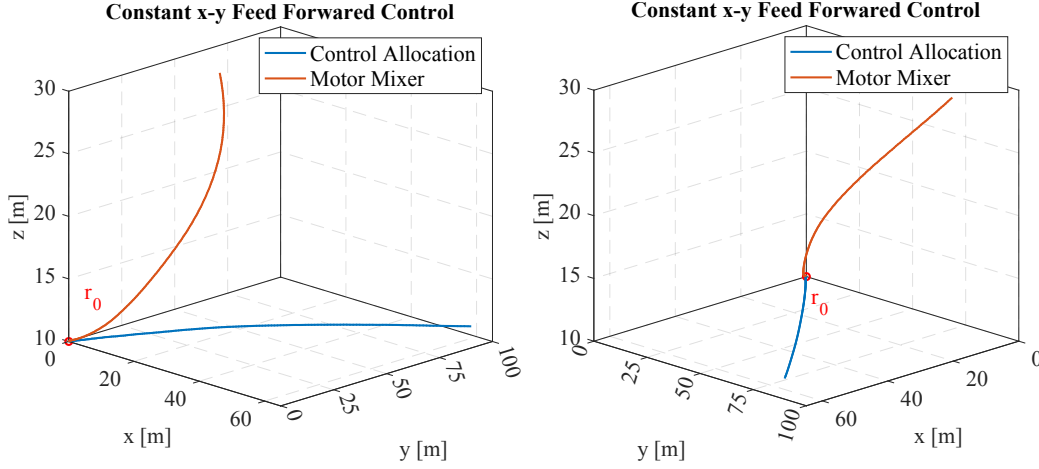


Figure 5.2: 3D-trajectory of a combined constant x_B and y_B feed forward control with the motor mixer and control allocation, given in two perspectives.

The incorrect motion is due to the fact that the motor mixer outputs the motor control commands in the form of PWM values. The PWM values are proportional to the engine rotation speeds ω_i . The motor mixer therefore adds the engine rotations for an x_B and y_B command for each individual engine instead of adding the forces that need to be generated as they are proportional to ω_i^2 . The linearity assumptions that are necessary for the motor mixer do not hold with the implemented motor mixer. For quadcopters, for example, linearity assumptions for the motor mixer can hold, since the changes in the motor turning velocities for a desired motion can be held small enough. This motor mixer, however, has a more complex case specific logic where individual engines need to be actuated in certain intensities. PX4 outputs the motor mixer signal als PWM values by default and does not allow easily to modify the output.

5.1.2 Actuation Effectiveness

For an energy-efficient operation of the airship, it is necessary to examine the actuation effectiveness of the propulsion system for specific motions and orientations. The knowledge about the axis with the most effective force and torque generation can be used as a constraint in the controller to enable more energy-efficient flight. The effectiveness in this case is measured by how much thrust the actuated engines have to generate in total to produce a desired resulting force. The effectiveness therefore refers to a relative effectiveness a of desired force to generated forces in total, given by

$$\eta_{\text{rel}} = \frac{\|\mathbf{F}_d\|_2}{\sum_i \|\mathbf{F}_i\|_2}, i = 1, \dots, 9, \quad (5.2)$$

where η_{rel} denotes the relative effectiveness, $\mathbf{F}_d$ denotes the desired force, and $\mathbf{F}_i$ denotes the force produced by engine i . The tests to determine the respective η_{rel} are carried out computationally in Octave [EatonEtAl20]. For a desired force $\mathbf{F}_d$ or a desired torque $\boldsymbol{\tau}_d$, the produced forces $\mathbf{F}_i$ and torques $\boldsymbol{\tau}_i$ are computed for the motor mixer and for the control allocation. $\mathbf{F}_d$ and $\boldsymbol{\tau}_d$ are normalized and applied around the airship in 1000 iterations per axis pair, so that a resulting force in every direction is covered. For example, for the effectiveness of a force around the x_B - y_B -axis, the force is varied per iteration with

$$\mathbf{F}_d = \begin{pmatrix} \cos \frac{2\pi k}{1000} \\ \sin \frac{2\pi k}{1000} \\ 0 \end{pmatrix}.$$

As described in the section above, the motor mixer in PX4 leads to an incorrect actuation of the engines due to the fact it forms a sum of the desired turning velocities ω_i of the individual engines instead of the desired forces. To allow for a comparison with the control allocation, a hypothetical motor mixer is set up such that it adds up the desired forces of the individual engines. This implementation is possible since the tests are carried out computationally in Octave instead of experimentally in PX4. The results for each axis pair are shown in Figure 5.3.

As expected, the control allocation is able to maintain a higher actuation effectiveness in most of the cases. In cases, where the resulting force or torque points to only one axis, the effectiveness of both are equivalent, like for the effectiveness for a force along the x_B - y_B -axis. The maximum effectiveness is given with 83 % when the generated force points only in the x_B -direction. In other cases, like for a force along the y_B - z_B -axis, the control allocation is able to even double the efficiency w.r.t. the motor mixer in some orientations.

For the control allocation, the highest efficiencies can be reached in scenarios where the stern propeller is actuated the most. In the example of the force along the x_B - z_B -axis, a feed forward control in the positive x_B -direction actuates the stern propeller along with several side engines to compensate for a downward pitch due to the low COG, as shown in Appendix A.1. The stern propeller is directly orientated in x_B -direction, while the side engines have a force component in every direction. In comparison, for a feed forward control in the z_B -direction, the force components in x_B - and y_B -direction of the actuated side engines are mutually cancelled out.

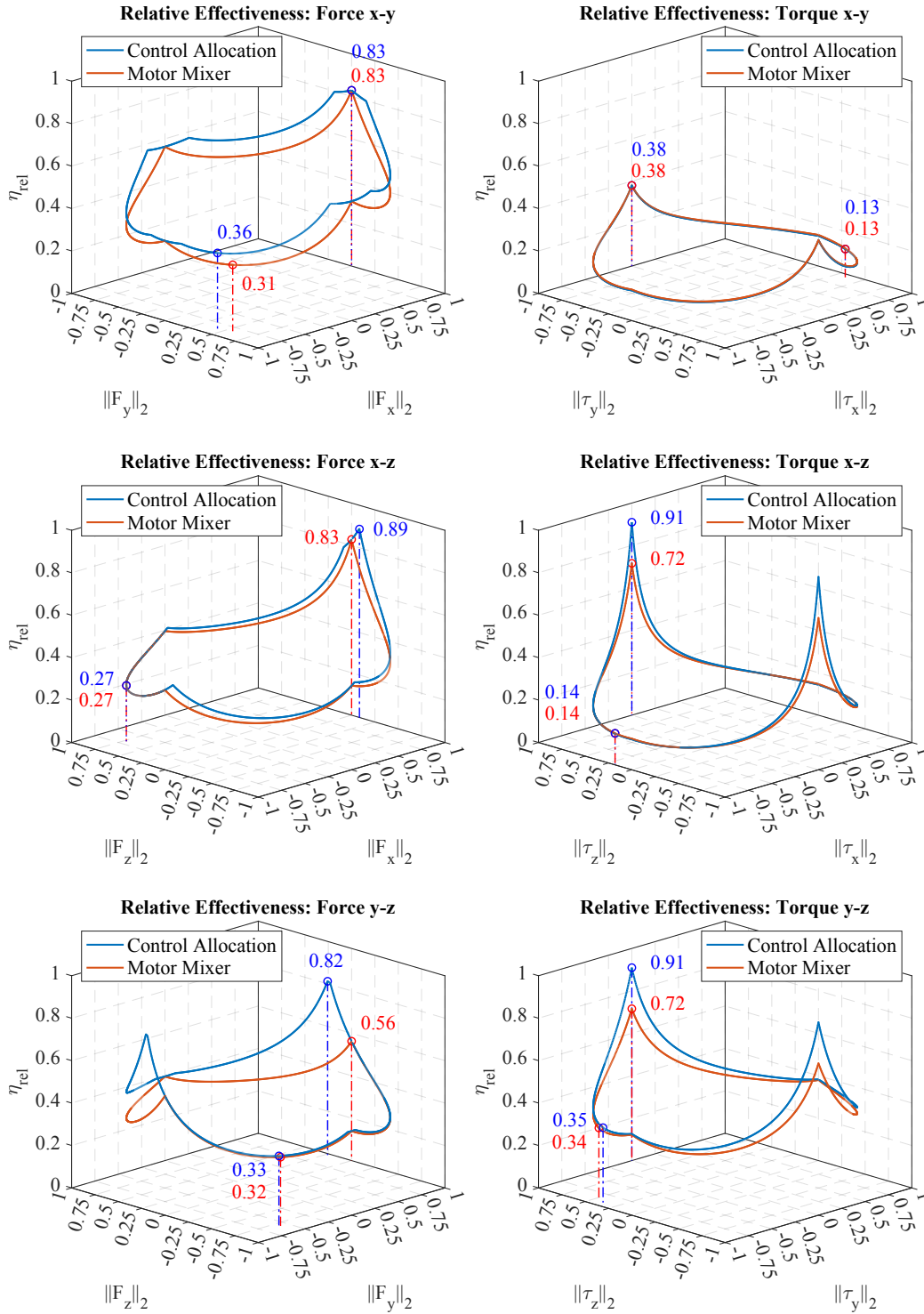


Figure 5.3: Distribution of the relative actuation effectiveness η_{rel} of the motor mixer and control allocation for a pair-wise combined feed forward control. η_{rel} is given by the ratio of a normalized force or torque in a desired direction to the sum of the forces and torques generated by the engines.

It can also be seen that a torque along the x_B -axis is highly inefficient. The small lever arm from the side engines to the COG are small so that only an efficiency of maximum 15 % can be reached and most of the energy is wasted. However, the low centered COG and inertia along the x_B -axis ensure an inherent stability of the system so that a control of the roll is not necessary in most scenarios. For a torque around the y_B - and z_B -axis, the large lever arm of the engines to the COG enable a more efficient application of the torque.

5.2 Validation of the Flight Controller

Now that the control allocation is validated and it is made sure that a desired control command $\mathbf{u}_d$ leads to a desired motion of the vehicle, the flight controller as a whole can be validated. Generally speaking, the task of the flight controller is to minimize the discrepancy between a trajectory point and the current state. Therefore, in this section, the ability of the flight controller to fly to given waypoints and to compensate for disturbances are evaluated in two parts.

In the first part, the vehicle is launched at the origin, $\mathbf{r}_0 = (0, 0, 0)^\top$, and the following five waypoints are given in the inertial frame I :

$$\mathbf{r}_1 = \begin{pmatrix} 0 \\ 0 \\ 10 \end{pmatrix}, \quad \mathbf{r}_2 = \begin{pmatrix} 0 \\ 50 \\ 10 \end{pmatrix}, \quad \mathbf{r}_3 = \begin{pmatrix} 50 \\ 50 \\ 10 \end{pmatrix}, \quad \mathbf{r}_4 = \begin{pmatrix} 50 \\ 0 \\ 10 \end{pmatrix}, \quad \mathbf{r}_5 = \mathbf{r}_1$$

The airship is supposed to fly to the waypoints in the given order. A waypoint is considered to be reached, when the distance between the current position to the desired waypoint position is less than 0.5 m:

$$\|\mathbf{r}_d - \mathbf{r}\|_2 \leq 0.5. \quad (5.3)$$

The same task is also performed by only using the position controller and setting a fixed yaw angle to validate if flight controller is capable of omnidirectional flight. In the second part, the ability to recover from a disturbance is examined by applying a force once while maintaining a position and once during the waypoint tracking task as described above.

5.2.1 Waypoint Tracking

Following waypoints is a common task. Waypoints can occur either in the form a destination point or trajectory points of a discretized trajectory. The flight controller is tested in a waypoint tracking task as described in the previous subsection. When using the whole control pipeline, the position controller minimized the position error and sets a yaw setpoint ψ_d for the attitude controller. The yaw setpoint is computed from the last waypoint $\mathbf{r}_{i-1}$ to the current waypoint $\mathbf{r}_i$ so that the vehicle always points to the current waypoint, as stated in Equation 4.17. The roll and the pitch angles are also controlled but are supposed to remain level.

The results of the waypoint tracking tasks with the whole control pipeline and with a fixed yaw angle are given in Figures 5.4, 5.5 and 5.6. To simplify readability, the waypoint tracking task with the yaw setpoint is referred to as the first run and the task with a fixed yaw angle is referred to as the second run.

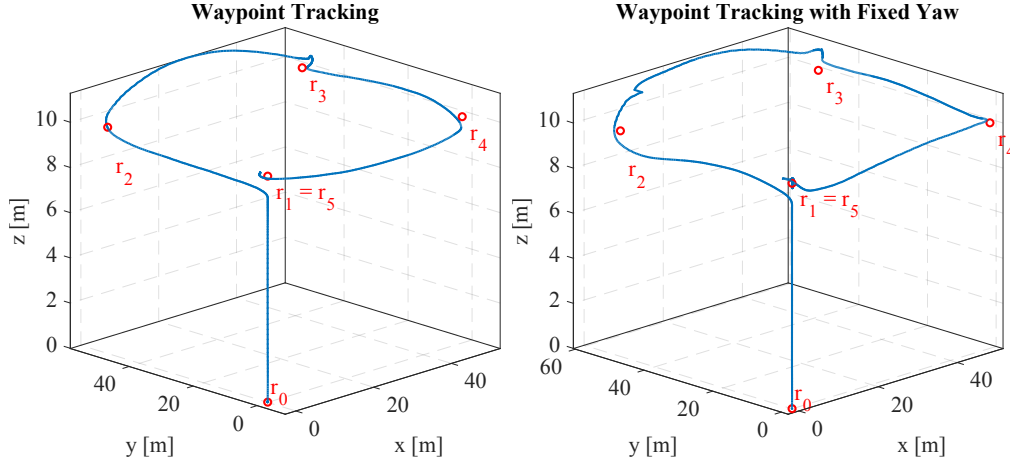


Figure 5.4: 3D-trajectories of the waypoint tracking task from $\mathbf{r}_0$ to $\mathbf{r}_5$, with the flight controller in the first run (left), and the same task with the flight controller, but a preset fixed yaw angle in the second run (right).

The trajectory of the flight controller in the first run is overall smoother and faster than in the second run. There is an overall more direct and smoother approach towards the subsequent waypoint in the first run, but the controller overshoots more for the control of the y -direction compared to the trajectory of the second run. In both cases, the roll and pitch angles are oscillating due to the pendulum effect of the low COG. The roll angle in particular has a strong oscillation in the first run which is always triggered when the airship reaches a waypoint and adjusts the yaw angle. In general, the yaw rotation always causes a roll rotation of the airship due to the low COG. The control allocation compensates for the resulting roll rotation, but cannot completely eliminate it.

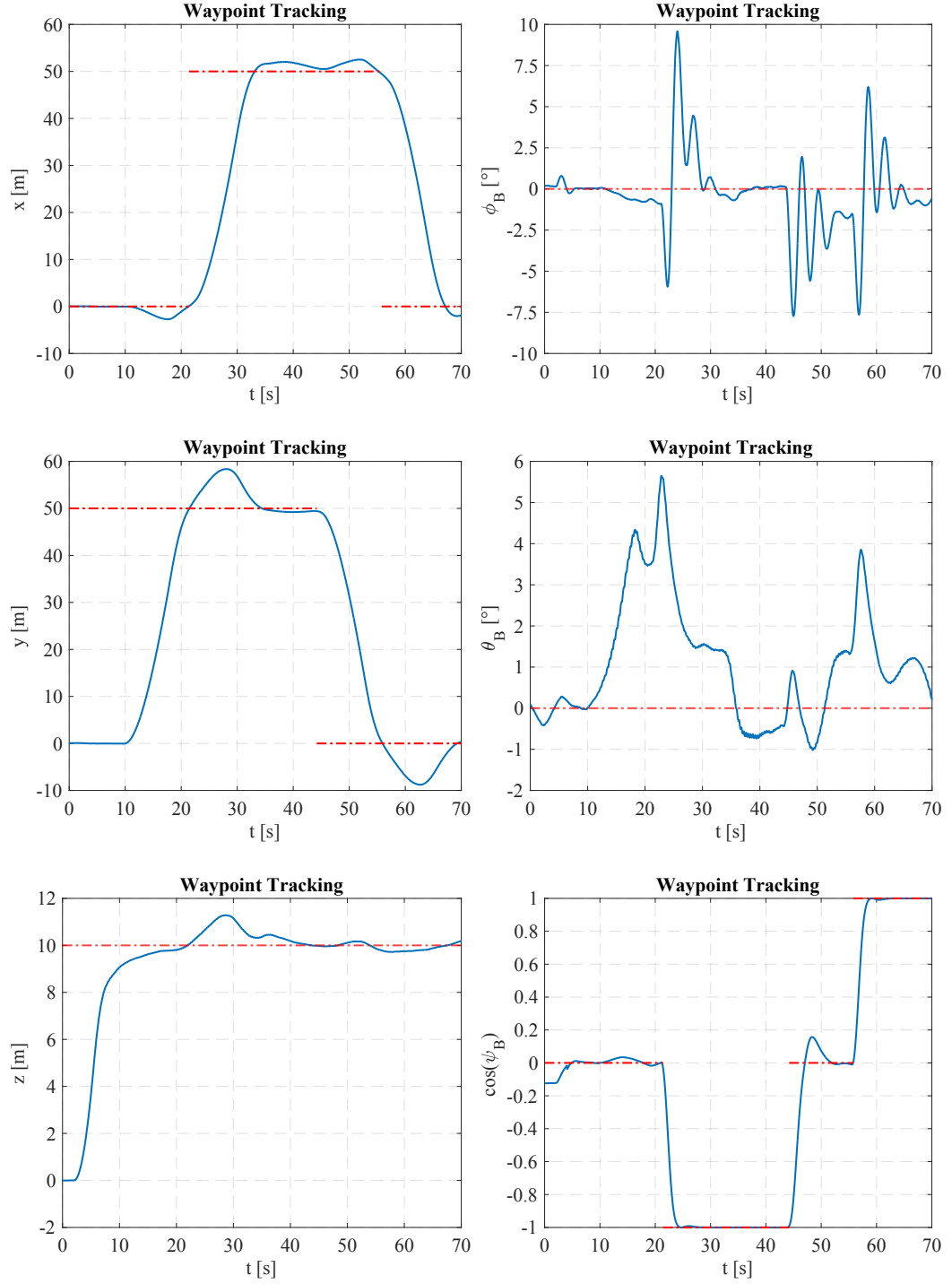


Figure 5.5: Trajectory of the waypoint tracking task from $\mathbf{r}_0$ to $\mathbf{r}_5$ with the flight controller in the first run. The red dash-dotted lines indicate the target state at the given time.

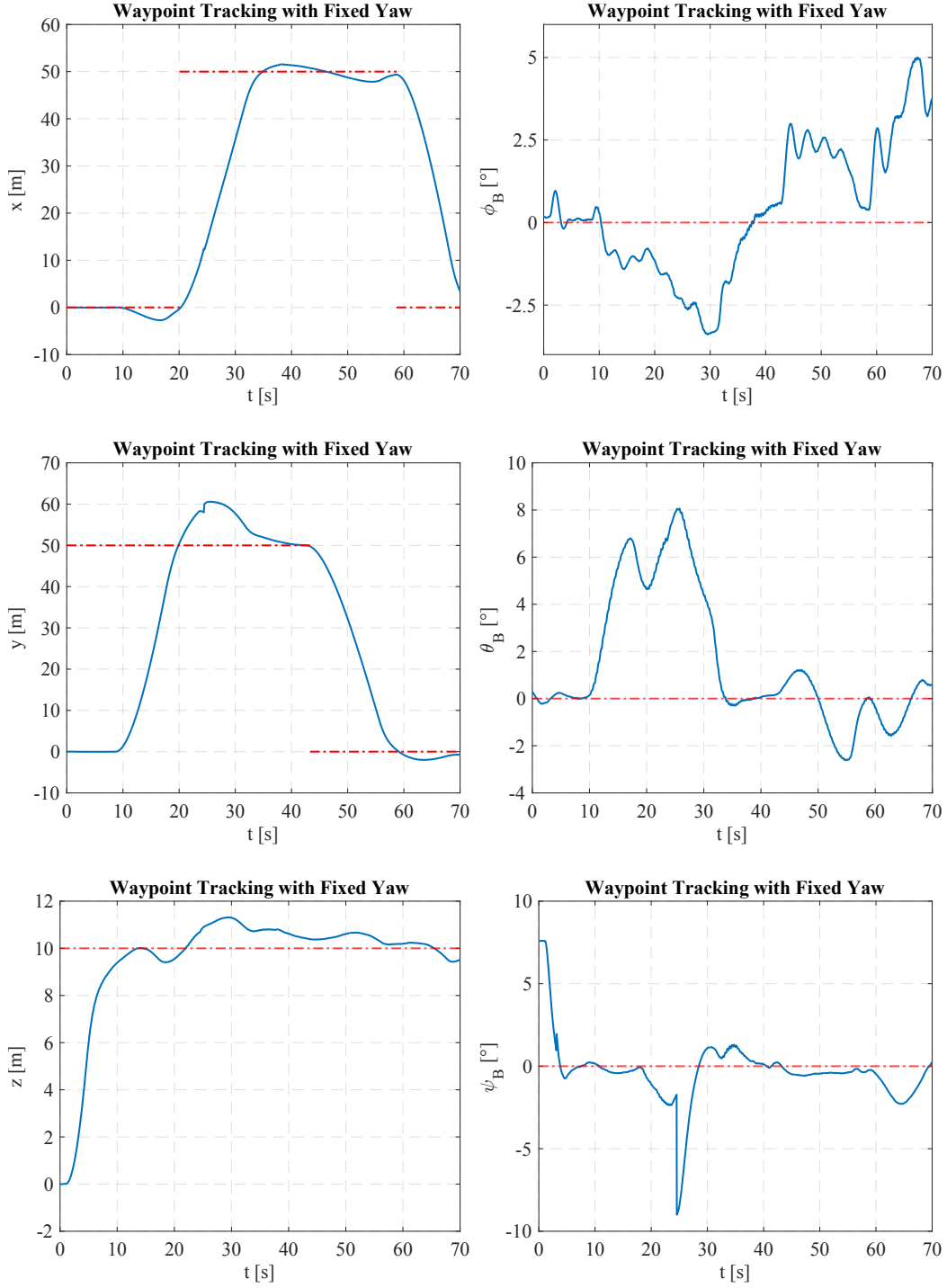


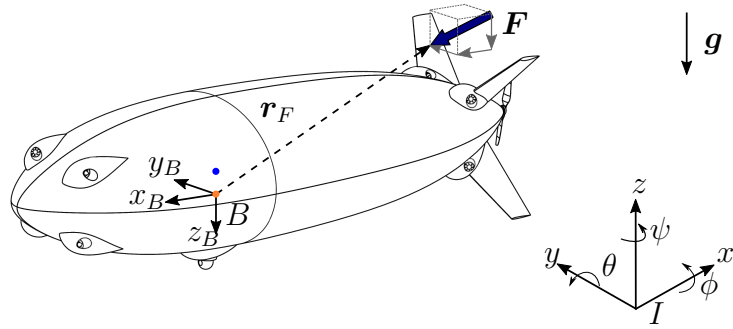
Figure 5.6: Trajectory of the waypoint tracking task from $\mathbf{r}_0$ to $\mathbf{r}_5$ with the flight controller, but a preset fixed yaw angle in the second run. The red dash-dotted lines indicate the target state at the given time.

5.2.2 External Force Compensation

In a final evaluation, it is examined how resistant the flight controller reacts against external disturbances and how long it takes to recover from it. For this purpose, an external force is applied to the airship in two scenarios. In the first scenario, the airship maintains a position 10 m above the ground and the following force is applied:

$$\mathbf{F} = \begin{pmatrix} 7000 \\ 7000 \\ 7000 \end{pmatrix} \text{ N}$$

$$\mathbf{r}_F = \begin{pmatrix} -3.6 \\ 0.6 \\ -0.6 \end{pmatrix} \text{ m.}$$



$\mathbf{r}_F$ denotes the position of the applied force $\mathbf{F}$, both given in the body frame B . The position of the force is at the SUL fin of the airship and the force acts perpendicular to the surface of the fin. This particular position for the force is chosen to apply an eccentric force that also causes the airship to spin and has a large lever arm. The force $\mathbf{F}$ is applied through the graphical interface of Gazebo and the duration is one Gazebo simulation time step, which equals 1 ms in real time. Due to the short duration of the applied force, the magnitude of the force has to be set high enough to cause a significant acceleration of the vehicle.

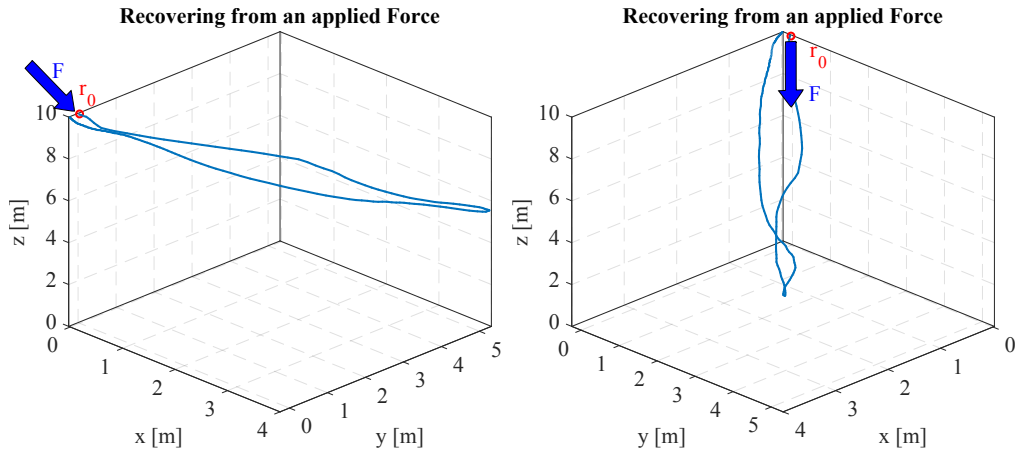


Figure 5.7: 3D-trajectory after the application of the force $\mathbf{F}$, given in two perspectives. $\mathbf{r}_0$ represents the starting and target position.

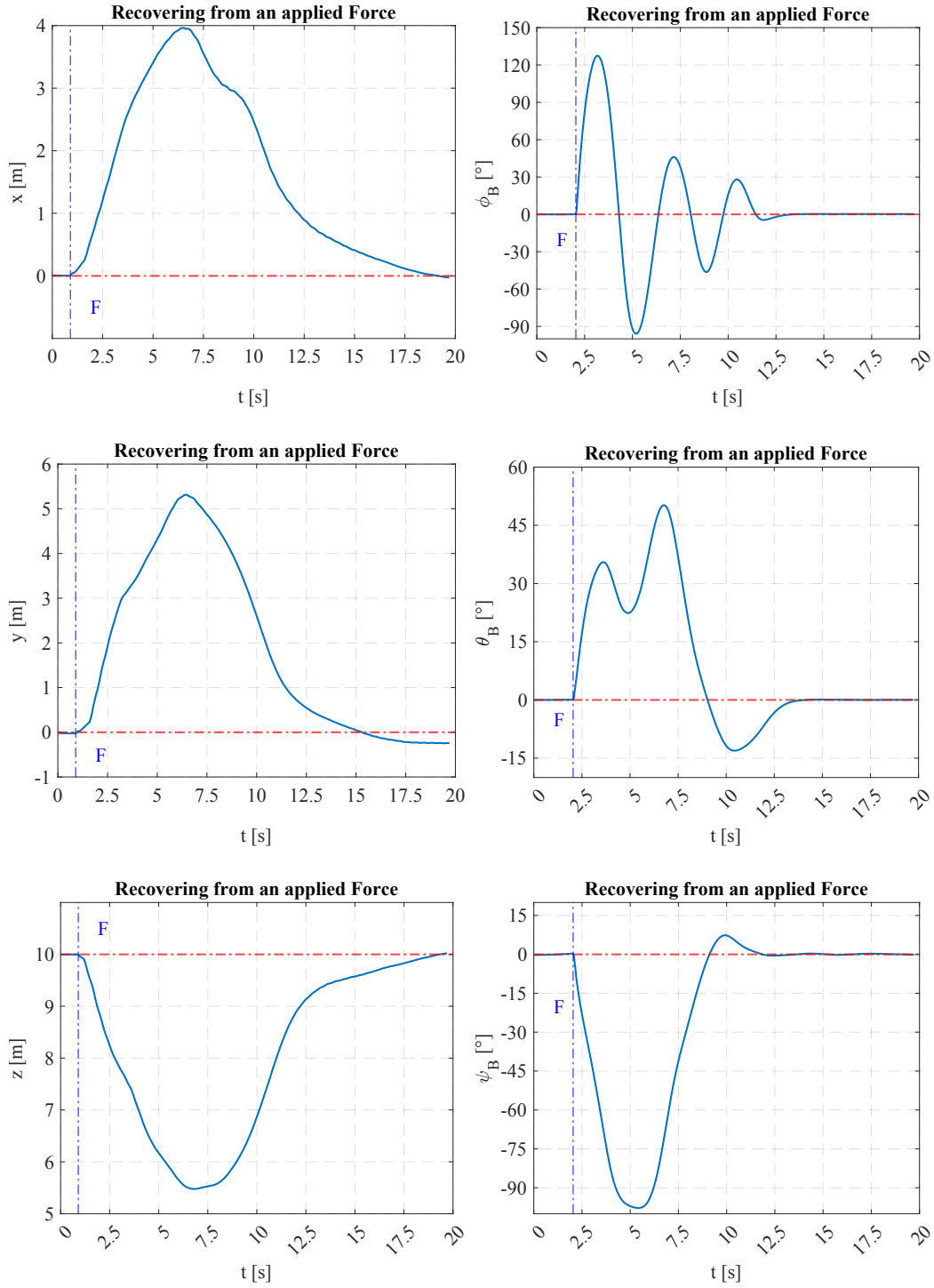


Figure 5.8: Trajectory after the application of the force $\mathbf{F}$. The red dash-dotted lines indicate the target state and the blue dash-dotted line indicates the time when $\mathbf{F}$ is applied.

The result of the first static application of the force is displayed in Figures 5.7 and 5.8. In this first scenario, the vehicle is launched 10 m above the ground and the given force $\mathbf{F}$ is applied at the local position $\mathbf{r}_F$.

The force causes the vehicle to swing out in each direction with a maximum of about 5.3 m in the y -direction, but the airship manages to get into a stable state. The orientation of the airship is completely restored after about 13 s and the position after about 18.5 s. The roll and the pitch are oscillating due to the pendulum effect of the low COG. The attitude controller slightly oversteers, whereas the position controller tends to understeer.

In the second scenario, the same force is applied on the vehicle while performing the waypoint tracking task of Section 5.2.1. $\mathbf{F}$ is applied between the second and the third waypoint, $\mathbf{r}_2$ and $\mathbf{r}_3$. The results are given in Figures 5.9 and 5.10.

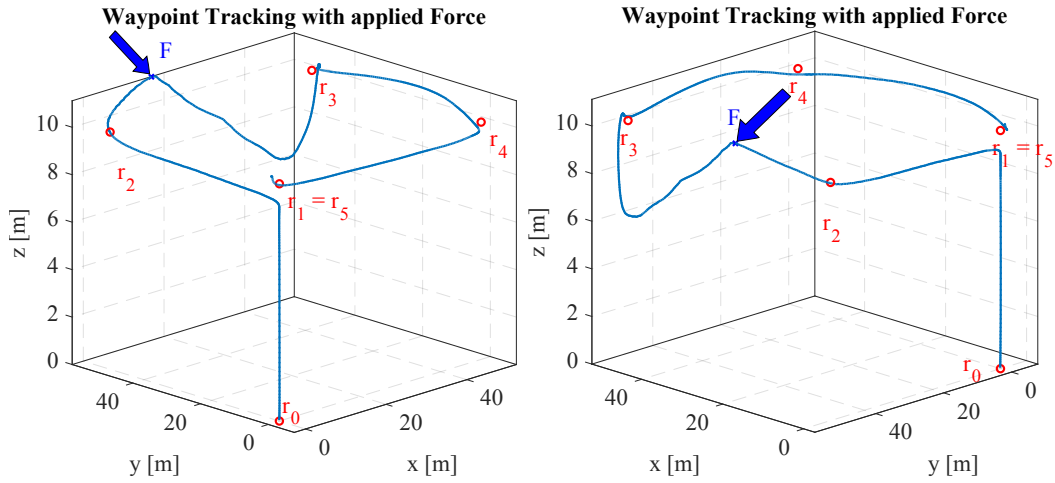


Figure 5.9: 3D-trajectory of the waypoint tracking task from $\mathbf{r}_0$ to $\mathbf{r}_5$ with the application of the force $\mathbf{F}$, given in two perspectives. The blue arrow indicates the position and the direction of $\mathbf{F}$.

Unlike in the first scenario, the applied force has a small effect on the movement in the x - and y -direction. For the translation in the z -direction and the rotation along the respective axes, the controller responses shown in Figure 5.10 are similar to the responses shown in figure 5.8, especially in the section after the impact of the force until recovery. The vehicle manages to recover in about the same time as in the first scenario and is able to complete the task. Despite the disturbance, the last waypoint was reached in about the same time as without the disturbance.

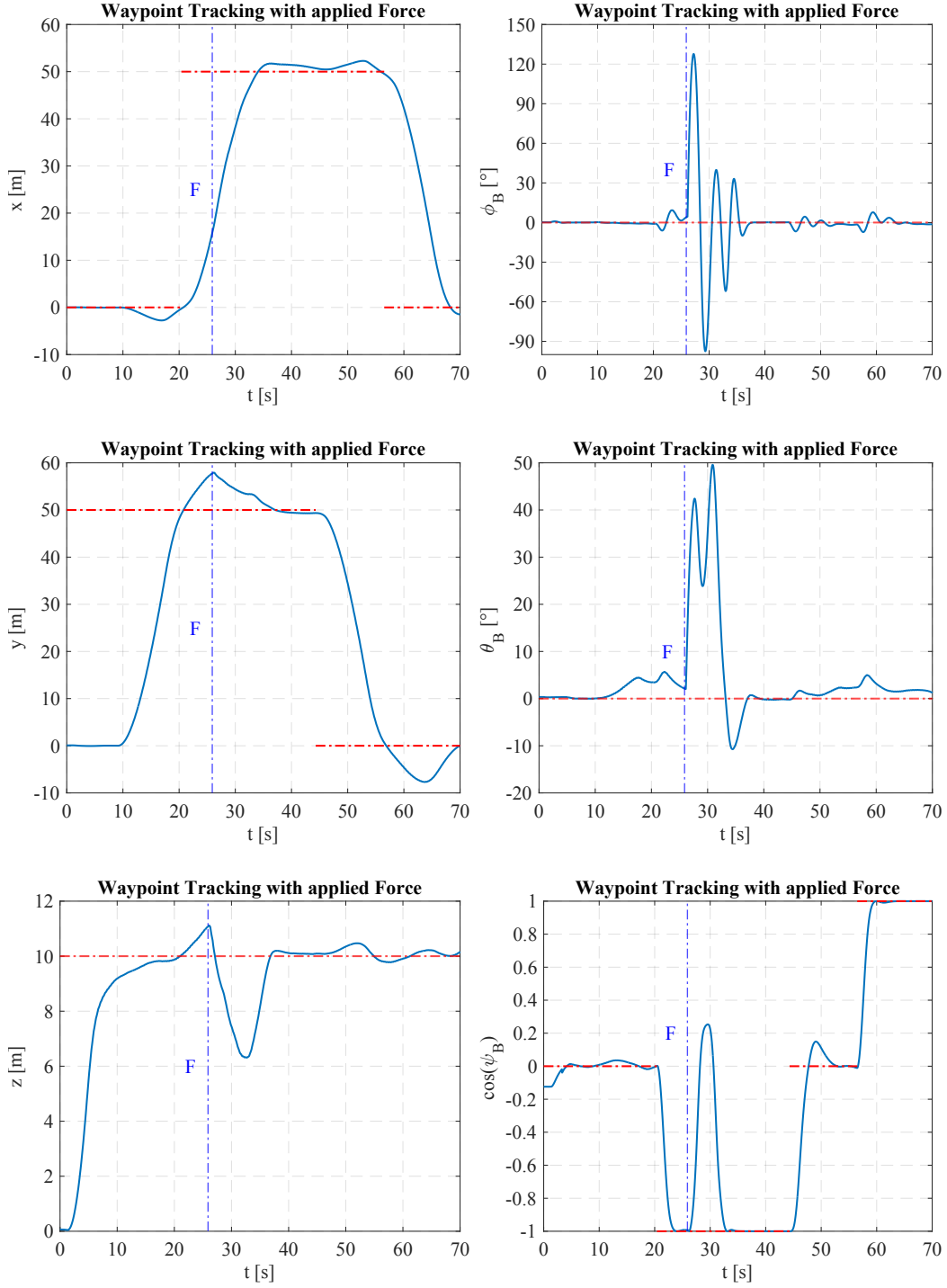


Figure 5.10: Trajectory of the waypoint tracking task from $\mathbf{r}_0$ to $\mathbf{r}_5$ with the application of the force $\mathbf{F}$. The red dash-dotted lines indicate the target state and the blue dash-dotted line indicates the time when $\mathbf{F}$ is applied.

Chapter 6

Conclusion

In this thesis, a flight controller has been developed for an omnidirectional airship drone with a novel propulsion system that allows for an instant maneuverability in every direction. The propulsion system consists of a stern engine, four rudders and eight side engines that are aligned and oriented in such a way that combinations of these are able generate a force and a torque in any direction. The airship thus has 6 degrees of freedom (DOF) and does not require an initial velocity initiate a motion in any direction, unlike with conventional propulsion systems. In the scope of this thesis, the rudders were not considered for the flight controller.

The development of the flight controller took place in five steps. **First**, a software platform has been selected that runs the autopilot and manages the tasks for the flight controller. Various flight stacks have been evaluated based on the requirements and compatibility with the flight control hardware. The PX4 flight stack has been selected on which the flight controller and the simulation have been implemented. In order to set up a simulation, the dynamics of the airship has been modeled in a **second step** with an emphasis on the physical, aerodynamic and motor model. The models were set up based on a CAD model of the airship and preliminary studies of the components of the airship. This dynamic model was then integrated into the simulation environment in a third step. The simulation was set up as a Software-In-The-Loop (SITL) in the Gazebo simulator, which enables the testing of the whole PX4 software pipeline through interaction with the simulation. With the SITL set up, the flight controller could then be developed and tested. The flight controller consists of two major parts, which are the control algorithm and the control allocation. The control allocation translates the vehicle control commands from the control algorithm to lower level motor control commands in to address the individual engines and to provide a simplified control interface for the control algorithm. Therefore, in a **fourth step**, a static and a dynamic allocation have been developed. The static motor mixer is primarily used for manual control and can be used for autonomous control under certain

circumstances. This circumstances include an assumption where the vehicle control command can be mapped linearly to the motor control command. Since this assumption does not hold for this propulsion system, the logic of the motor mixer also does not hold anymore when the vehicle control command contains more than one control variable. Therefore, a dynamic control allocation has been developed for the autopilot that allows for a more precise and energy-efficient allocation. In the **final step**, a cascaded P-PID position and attitude controller has been implemented, consisting of a P position control and PID velocity control in the case of the position controller, and also a P attitude control and PID angular velocity control for the attitude controller.

The flight controller and its components have been tested in various flight tasks. First, the motor mixer and the control allocation have been validated for their accuracy to generate a desired motion of the vehicle, and for their effectiveness in regards to the amount of thrust the individual engines have to produce in order to result in a desired force. The accuracy for both the motor mixer and control allocation are on average about 95 %, which indicates that a motion in one direction causes a 5 % deviation on average in other directions. The control allocation, however, is slightly more accurate since it considers the dynamics of the system, while several assumptions have been made for the motor mixer.

The limitation of the motor mixer, however, is reached when more than one direction is to be controlled. The mixing logic does not hold anymore, since the mixer in PX4 sums up the turning velocities of the engines instead of the forces that have to be generated by the individual engines. The generated force, however, is proportional to the square of the turning velocity of an engine. Therefore, the relation is non-linear. The complex mixing logic of this propulsion system, as presented in Appedix A.1, does not allow for linearity assumptions in the operation of the engines. The control allocation does not have this issue, since it adds up the desired forces and allocates these to the respective engines in the most energy-efficient way. The control allocation computes the optimal allocation of a desired force and torque to the engines at each time step by setting up a linear optimization problem and computing the optimal solution with the simplex method. This is also a major advantage compared a static motor mixer that adds up the desired forces, since the motor mixer can reach an effectiveness of up to 91 %, whereas the motor mixer can only reach a maximum of 83 %. The lowest maximum effectiveness with 14 % for both is achieved with a torque around the roll axis. The difference increases as more directions are controlled at the same time.

Another examined advantage of the control allocation is the possibility to compensate for failed engines. In theory, six of the nine engines should suffice for a motion in any of the 6 DOF. When a motor failure is detected, the linear optimization problem could be adjusted so that the other engines compensate for the

failed one. In a dedicated test, however, a solution to the optimization problem, as stated in Section 4.3.1, could not be found in such a scenario. This is probably due to the formalization of the linear optimization problem. A tolerance to the constraints of the problem could lead to a solution by searching a near-optimal solution with the lowest cost instead of a unique optimal solution.

In the validation of the flight controller, the airship performs two types of tasks. In the first type, five waypoints are tracked, and in the second, a force is applied in order to evaluate the capability of the flight controller to fly to any given point and to compensate for external disturbances along the way. The flight controller was able to perform both tasks. The given waypoints could be flown using the whole control pipeline, but also with a fixed yaw angle to test and demonstrate the ability for omnidirectional flight. Furthermore, a force acting on airship's stern-up-left rudder could be compensated for in a scenario where the airship has to maintain a fixed position and during the waypoint tracking task.

In a future work, a next step would be to validate the flight controller on the real airship system. A flight test would bring additional challenges that have not been represented in the simulation so far. The aerodynamic model of the airship is only represented by a damping, depending on the velocity of the airship. The large surface of the airship compared to its mass makes it vulnerable to wind. Also, the lift generated during flight by an upward pitch should not be neglected. Improving the aerodynamic model and identifying model parameters in manual flight tests could therefore be a part in a future work. With energy-efficiency as one of the main goals, a future work could also include the consideration of the rudders in the flight controller. For instance, actuating the stern propeller and regulating the attitude with the rudders in a straight flight could increase the actuation effectiveness from 83 % to nearly 100 %. A scenario could be conceivable where this particular actuation configuration is used in a cruise mode for straight flight and waypoint tracking in order to maximize energy-efficiency and flight time. The side engines could be used in a hover, precision or stabilizing mode where the side engines could enable a precise movement.

To sum it up, first, various flight stacks have been evaluated and the PX4 flight stack has been selected for the flight controller. Then, the dynamic model of the airship has been created containing a physical, aerodynamic and motor model. The dynamic model has been implemented in a simulation environment in the Gazebo simulator as a SITL. Based on the dynamic model of the airship, a motor mixer for manual control and a control allocation for autonomous control have been developed for the over-actuated system by stating and solving the allocation as linear optimization problem. Finally, a cascaded P-PID position and attitude controller has been implemented. The flight controller has been successfully evaluated in several scenarios and flight tasks, validating its performance, resilience and the ability for omnidirectional flight.

Appendices

A.1 Motor Mixer

The derivation of the motor mixer is elaborated in Section 4.2. Since the system is over-actuated, there are infinite possibilities to actuate the engines in order to achieve a desired motion. The task of the motor mixer is to simplify this logic for manual and, under certain circumstances, for autonomous control. A desired vehicle control command $\mathbf{u}_d$ is translated to an actuator control command $\mathbf{u}_{\text{act}}$. An optimal motor mixer is computed by solving the linear optimization problem of Equation 4.5, as described in Section 4.2.1. The resulting optimal motor mixer is then given by

$$\mathbf{B}_{\text{mix}} = \begin{bmatrix} | & | & \dots & | \\ \mathbf{b}_x & \mathbf{b}_y & & \mathbf{b}_\psi \\ | & | & & | \end{bmatrix}, \quad (\text{A.1})$$

where the respective optimal actuation vectors $\mathbf{b}_{(\cdot)}$ for a feed forward control in the respective directions are derived from the solution of the linear optimization problem, as described in Section 4.2.1. The respective $\mathbf{b}_{(\cdot)}$ are given by

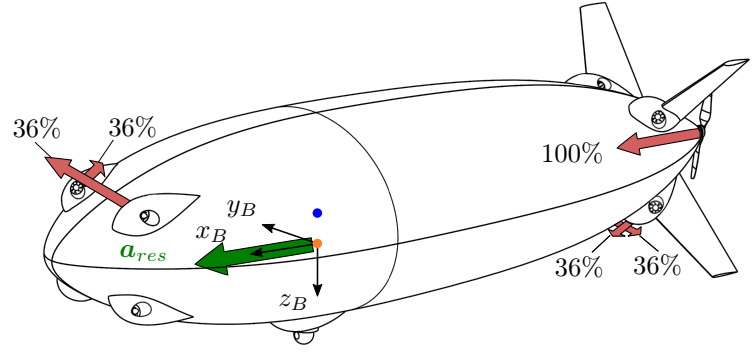
$$\mathbf{b}_{(\cdot)} = \begin{cases} u_{d,(\cdot)} \geq 0, & \mathbf{b}_{(\cdot)+} \\ u_{d,(\cdot)} < 0, & \mathbf{b}_{(\cdot)-} \end{cases}, \quad (\text{A.2})$$

where $u_{d,(\cdot)}$ denotes the corresponding entry of $\mathbf{u}_d$. $\mathbf{u}_{\text{act}}$ can be then computed by

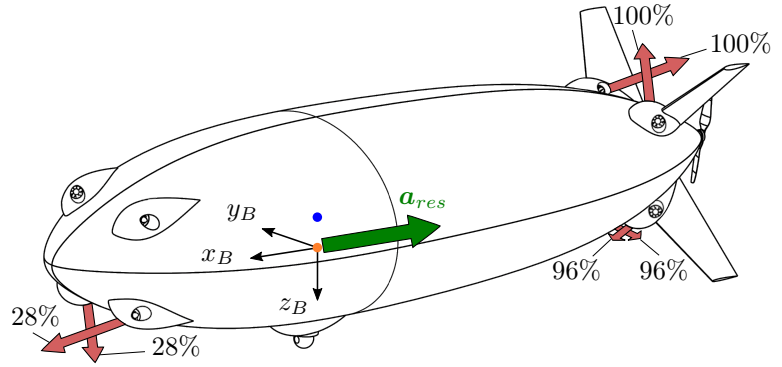
$$\mathbf{u}_{\text{act}} = \mathbf{B}_{\text{mix}} \mathbf{u}_d. \quad (\text{A.3})$$

If, for example, $\mathbf{u}_d$ consists of only one value for the positive x_B -direction, then $\mathbf{u}_{\text{act}} = \mathbf{b}_{x+}$. In the following, the respective $\mathbf{b}_{(\cdot)}$ listed and visualized for each controllable direction.

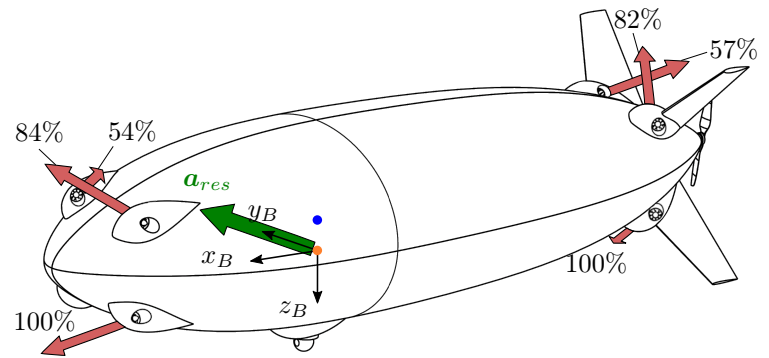
$$\mathbf{b}_{x+} = \begin{pmatrix} 0.3597 \\ 0.3597 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0.3597 \\ 0.3597 \\ 1 \end{pmatrix}$$

Figure A.1: Translation in positive x_B -direction

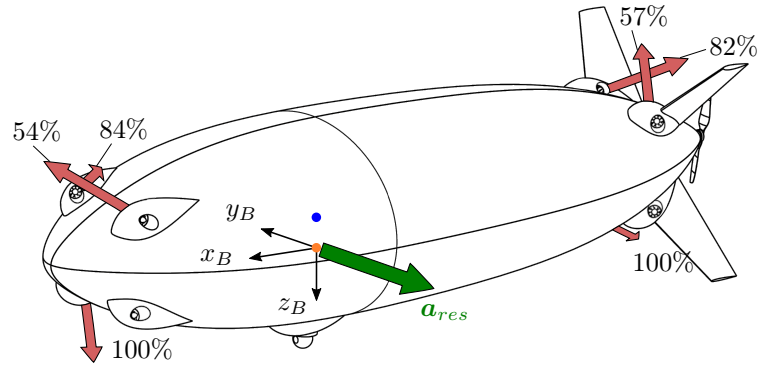
$$\mathbf{b}_{x-} = \begin{pmatrix} 0 \\ 0 \\ 0.2792 \\ 0.2792 \\ 1 \\ 1 \\ 0.9602 \\ 0.9602 \\ 0 \end{pmatrix}$$

Figure A.2: Translation in negative x_B -direction

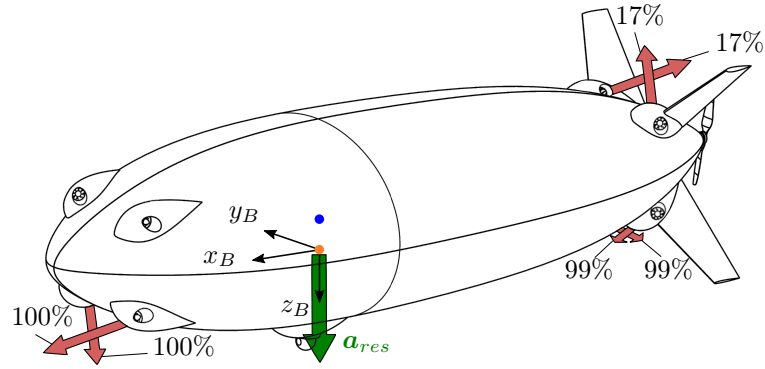
$$\mathbf{b}_{y+} = \begin{pmatrix} 0.84 \\ 0.5422 \\ 0 \\ 1 \\ 0.82 \\ 0.5724 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

Figure A.3: Translation in positive y_B -direction

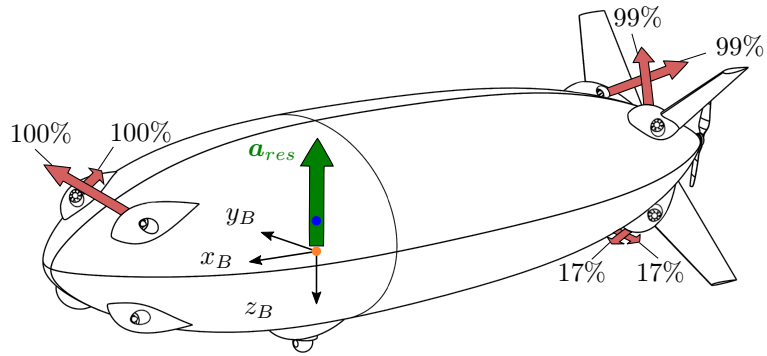
$$\mathbf{b}_{y-} = \begin{pmatrix} 0.5422 \\ 0.84 \\ 1 \\ 0 \\ 0.5724 \\ 0.82 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Figure A.4: Translation in negative y_B -direction

$$\mathbf{b}_{z+} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 0.1678 \\ 0.1678 \\ 0.9858 \\ 0.9858 \\ 0 \\ 0 \end{pmatrix}$$

Figure A.5: Translation in positive z_B -direction

$$\mathbf{b}_{z-} = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0.9858 \\ 0.9858 \\ 0.1678 \\ 0.1678 \\ 0 \\ 0 \end{pmatrix}$$

Figure A.6: Translation in negative z_B -direction

$$\mathbf{b}_{\phi+} = \begin{pmatrix} 1 \\ 0 \\ 0.9649 \\ 0.2626 \\ 0.9649 \\ 0.2626 \\ 1 \\ 0 \\ 0 \end{pmatrix}$$

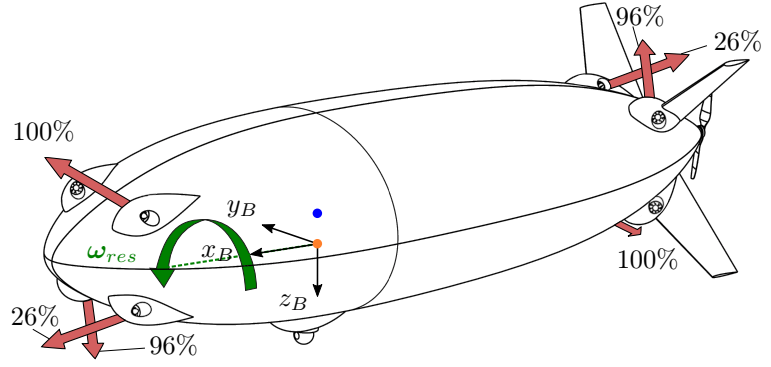


Figure A.7: Rotation around positive roll-axis

$$\mathbf{b}_{\phi-} = \begin{pmatrix} 0 \\ 1 \\ 0.2626 \\ 0.9649 \\ 0.2626 \\ 0.9649 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

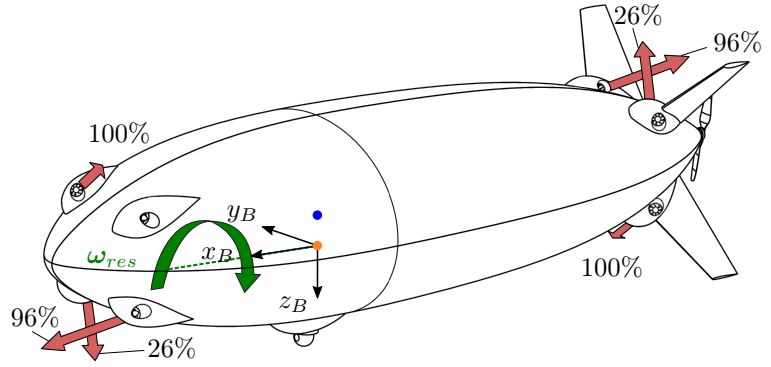


Figure A.8: Rotation around negative roll-axis

$$\mathbf{b}_{\theta+} = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

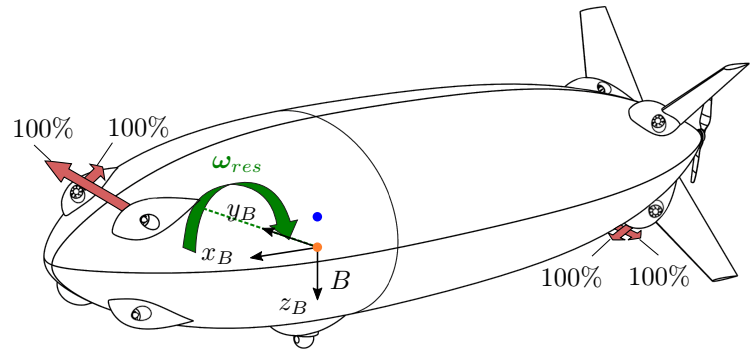


Figure A.9: Rotation around positive pitch-axis

$$\mathbf{b}_{\theta-} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

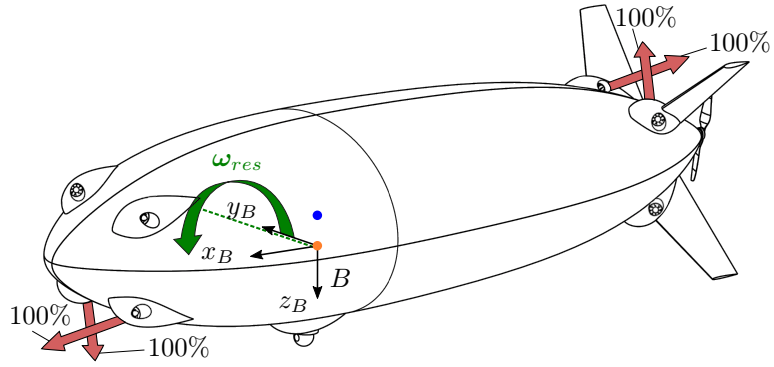


Figure A.10: Rotation around negative pitch-axis

$$\mathbf{b}_{\psi+} = \begin{pmatrix} 1 \\ 0 \\ 0.4553 \\ 0.8903 \\ 0.4553 \\ 0.8903 \\ 1 \\ 0 \\ 0 \end{pmatrix}$$

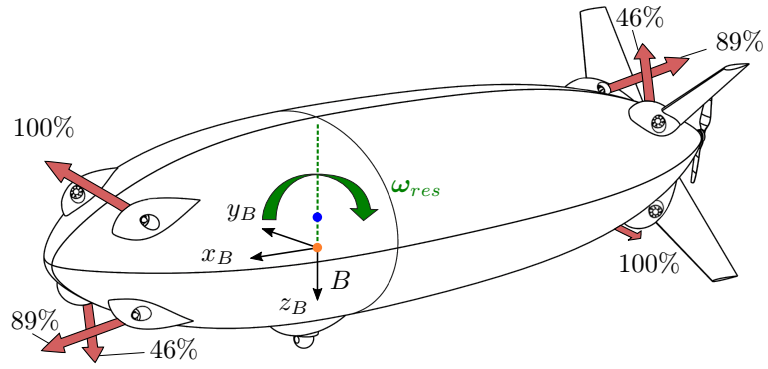


Figure A.11: Rotation around positive yaw-axis

$$\mathbf{b}_{\psi-} = \begin{pmatrix} 0 \\ 1 \\ 0.8903 \\ 0.4553 \\ 0.8903 \\ 0.4553 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

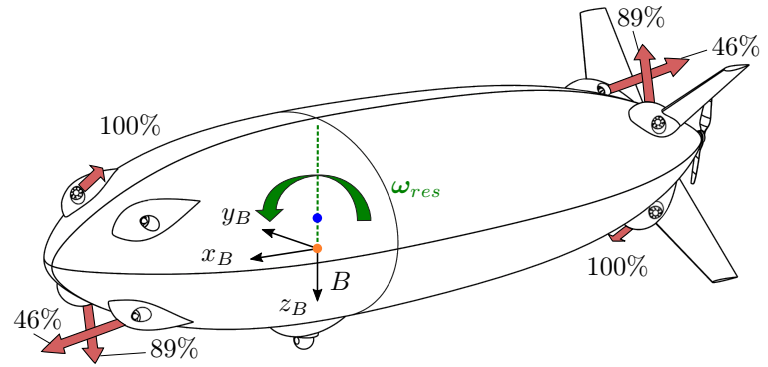


Figure A.12: Rotation around negative yaw-axis

Bibliography

- [AntoniouLu07] Antoniou, A.; sheng Lu, W.: Practical optimization: algorithms and engineering applications. Springer, 2007.
- [Ard22] ArduPilot flight stack, web page: <http://ardupilot.org/>, 2022.
- [Bodson02] Bodson, M.: Evaluation of Optimization Methods for Control Allocation. Journal of Guidance, Control, and Dynamics, Vol. 25, No. 4, pp. 703–711, 2002.
- [Boris22] Boris, B.: <https://betaflight.com>, accessed: 28-05-2022.
- [BruntonKutz19] Brunton, S.L.; Kutz, J.N.: Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control. Cambridge University Press, 2019.
- [BuenoEtAl02] Bueno, S.; Azinheira, J.; Ramos, J.; Paiva, E.; Rives, P.; Elfes, A.; Carvalho, J.; Silveira, G.: Project AURORA: Towards an autonomous robotic airship. Workshop on Aerial Robotics, IEEE International Conference on Intelligent Robots and Systems, 2002.
- [Cle22] Cleanflight Dev Team, "Cleanflight": <https://cleanflight.com>, accessed: 28-05-2022.
- [ConradShiakolasYih00] Conrad, K.L.; Shiakolas, P.S.; Yih, T.C.: In Robotic Calibration Issues: Accuracy, Repeatability and Calibration. 2000.
- [Cub22] Cube Pilot, "Cube Orange" flight controller: <https://cubepilot.com/#/cube>, accessed: 02-06-2022.
- [Dantzig63] Dantzig, G.B.: Linear programming and extensions. Princeton, N.J.: Princeton University Press, 1963.
- [Das22] Dassault Systèmes SOLIDWORKS Corp., "SolidWorks": <https://solidworks.com>, accessed: 28-05-2022.
- [Drela14] Drela, M.: Flight vehicle aerodynamics, 2014.

- [dro18] Drone industry insights - drones in the energy industry: The energy drone operator benchmark. <http://droneii.com>, 2018.
- [EatonEtAl20] Eaton, J.W.; Bateman, D.; Hauberg, S.; Wehbring, R.: GNU Octave version 6.1.0 manual: a high-level interactive language for numerical computations, 2020.
- [EbeidEtAl18] Ebeid, E.; Skriver, M.; Terkildsen, K.H.; Jensen, K.; Schultz, U.P.: A survey of Open-Source UAV flight controllers and flight simulators. *Microprocessors and Microsystems*, Vol. 61, pp. 11–20, 2018.
- [ElmokademSavkin21] Elmokadem, T.; Savkin, A.V.: Towards Fully Autonomous UAVs: A Survey. *Sensors*, Vol. 21, No. 18, 2021.
- [Fossen11] Fossen, T.: *Handbook of Marine Craft Hydrodynamics and Motion Control*. 2011.
- [FurrerEtAl16] Furrer, F.; Burri, M.; Achtelik, M.; Siegwart, R.: RotorS—A Modular Gazebo MAV Simulator Framework. In Koubaa, A. (Ed.): *Robot Operating System (ROS): The Complete Reference (Volume 1)*, pp. 595–625. Cham: Springer International Publishing, 2016.
- [GarcíaMolina19] García, J.; Molina, J.M.: Simulation in Real Conditions of Navigation and Obstacle Avoidance with PX4/Gazebo Platform. In 2019 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops), pp. 979–984, 2019.
- [GarpingerHägglundÅström14] Garpinger, O.; Hägglund, T.; Åström, K.J.: Performance and robustness trade-offs in PID control. *Journal of Process Control*, Vol. 24, No. 5, pp. 568–577, 2014.
- [GorjupLiarokapis20] Gorjup, G.; Liarokapis, M.: A Low-Cost, Open-Source, Robotic Airship for Education and Research. *IEEE Access*, Vol. 8, pp. 70713–70721, 2020.
- [GrechiCaiti16] Grechi, S.; Caiti, A.: Comparison between Optimal Control Allocation with Mixed Quadratic & Linear Programming Techniques. *IFAC-PapersOnLine*, Vol. 49, No. 23, pp. 147–152, 2016. 10th IFAC Conference on Control Applications in Marine Systems CAMS 2016.
- [HanoverEtAl22] Hanover, D.; Foehn, P.; Sun, S.; Kaufmann, E.; Scaramuzza, D.: Performance, Precision, and Payloads: Adaptive Nonlinear MPC for Quadrotors. *IEEE Robotics and Automation Letters*, Vol. 7, No. 2, pp. 690–697, 2022.
- [Her22] Herelink radio controller: <https://docs.cubepilot.org/user-guides/herelink/herelink-overview>, accessed: 02-06-2022.

- [HoughtonEtAl13] Houghton, E.; Carpenter, P.; Collicot, S.; Valentine, D.: *Aerodynamics for Engineering Students*. 2013.
- [HwangboEtAl17] Hwangbo, J.; Sa, I.; Siegwart, R.; Hutter, M.: Control of a Quadrotor With Reinforcement Learning. *IEEE Robotics and Automation Letters*, Vol. 2, No. 4, pp. 2096–2103, 2017.
- [InvernizziEtAl21] Invernizzi, D.; Giurato, M.; Gattazzo, P.; Lovera, M.: Comparison of Control Methods for Trajectory Tracking in Fully Actuated Unmanned Aerial Vehicles. *IEEE Transactions on Control Systems Technology*, Vol. 29, No. 3, pp. 1147–1160, 2021.
- [KoenigHoward04] Koenig, N.; Howard, A.: Design and Use Paradigms for Gazebo, An Open-Source Multi-Robot Simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 2149–2154, Sendai, Japan, 2004.
- [KoubaaEtAl19] Koubaa, A.; Allouch, A.; Alajlan, M.; Javed, Y.; Belghith, A.; Khalgui, M.: Micro Air Vehicle Link (MAVLink) in a Nutshell: A Survey. *CoRR*, Vol. abs/1906.10641, 2019.
- [LiNahonSharf11] Li, Y.; Nahon, M.; Sharf, I.: Airship dynamics modeling: A literature review. *Progress in Aerospace Sciences*, Vol. 47, No. 3, pp. 217–239, 2011.
- [LuenbergerYe21] Luenberger, D.G.; Ye, Y.: *Linear and Nonlinear Programming*, 2021.
- [Makhorin01] Makhorin, A.: Gnu linear programming kit (glpk). Department for Applied Informatics, Moscow Aviation Institute, Moscow, Russia, 2001.
- [MalekSedanMohdHarithuddin18] Malek, A.; Sedan, F.; Mohd Harithuddin, A.S.: Development of attitude control system for hybrid airship vehicle. *International Journal of Engineering and Technology(UAE)*, Vol. 7, pp. 99–106, 2018.
- [MarksWhidborneYamamoto12] Marks, A.; Whidborne, J.F.; Yamamoto, I.: Control allocation for fault tolerant control of a VTOL octorotor. In *Proceedings of 2012 UKACC International Conference on Control*, pp. 357–362, 2012.
- [Mav22] Mavlink Dev Team, "Mavlink": <https://mavlink.io>, accessed: 28-05-2022.

- [MazurEtAl16] Mazur, M.; Wisniewski, A.; McMillan, J.; et al.: Clarity from above: PwC global report on the commercial applications of drone technology. Warsaw: Drone Powered Solutions, PriceWater house Coopers, 2016.
- [McGheeBachmannZyda01] McGhee, R.; Bachmann, E.; Zyda, M.: Rigid Body Dynamics, Inertial Reference Frames, and Graphics Coordinate Systems: A Resolution of Conflicting Conventions and Terminology. 2001.
- [Musa18] Musa, S.: Techniques for Quadcopter Modelling & Design: A Review. Vol. 5, 2018.
- [NambaYamamotoKaneda97] Namba, R.; Yamamoto, T.; Kaneda, M.: Robust PID controller and its application. In 1997 IEEE International Conference on Systems, Man, and Cybernetics. Computational Cybernetics and Simulation, Vol. 4, pp. 3636–3641 vol.4, 1997.
- [Pap22] Paparazzi UAV Dev Team, "Paparazzi uav": <https://wiki.paparazziuav.org>, accessed: 28-05-2022.
- [PerezEtAl19] Perez, A.; Wagner, J.; Burggräf, P.; Roth, H.: Quadrotors in factory applications: design and implementation of the quadrotor's P-PID cascade control system Modeling and implementation. SN Applied Sciences, Vol. 1, 2019.
- [Pix22] Pixhawk flight controller hardware, "Pixhawk": <https://pixhawk.org>, accessed: 28-05-2022.
- [PodhradskýEtAl13] Podhradský, M.; Bone, J.; Coopmans, C.; Jensen, A.: Battery model-based thrust controller for a small, low cost multirotor Unmanned Aerial Vehicles. In 2013 International Conference on Unmanned Aircraft Systems (ICUAS), pp. 105–113, 2013.
- [PowersMellingerKumar15] Powers, C.; Mellinger, D.; Kumar, V.: Quadrotor Kinematics and Dynamics. Springer Netherlands, 2015.
- [PriceEtAl22] Price, E.; Liu, Y.T.; Black, M.J.; Ahmad, A.: Simulation and Control of Deformable Autonomous Airships in Turbulent Wind. In Ang Jr, M.H.; Asama, H.; Lin, W.; Foong, S. (Eds.): Intelligent Autonomous Systems 16, pp. 608–626, Cham: Springer International Publishing, 2022.
- [PX422] PX4-Dev team, "PX4 Documentation": <http://docs.px4.io>, accessed: 28-05-2022.
- [QGC22] QGC Dev Team, "QGroundControl": <http://qgroundcontrol.com>, accessed: 28-05-2022.

- [Sta22] Stanford Artificial Intelligence Laboratory et al.: Robotic Operating System, accessed: 28-05-2022.
- [Thomasson00] Thomasson, P.: Equations of Motion of a Vehicle in a Moving Fluid. *Journal of Aircraft - J AIRCRAFT*, Vol. 37, pp. 630–639, 2000.
- [ThrunBurgardFox05] Thrun, S.; Burgard, W.; Fox, D.: Probabilistic robotics. Cambridge, Mass.: MIT Press, 2005.
- [Vanderbei20] Vanderbei, R.J.: Linear Programming: Foundations and Extensions, 2020.

Erklärung

Hiermit versichere ich, dass

- ich die vorliegende Arbeit selbständig verfasst habe,
- ich keine anderen als die angegebenen Quellen benutzt und alle wörtlich oder sinngemäß aus anderen Werken übernommenen Aussagen als solche gekennzeichnet habe,
- ich die eingereichte Arbeit weder vollständig noch in Teilen bereits veröffentlicht habe,
- das elektronische Exemplar mit den anderen Exemplaren übereinstimmt.

Datum

Unterschrift